ASPIRE: An Agentic System-Identification Platform for Iterative Reasoning and Exploration

Erik Derner[a,*], Jiří Kubalík[a] and Robert Babuška[a,b]

[a]Czech Institute of Informatics, Robotics and Cybernetics, Czech Technical University in Prague, Jugoslávských partyzánů 1580/3, 160 00, Prague 6, Czech Republic

[b]Department of Cognitive Robotics, Delft University of Technology, Mekelweg 2, 2628 CD, Delft, The Netherlands

ARTICLE INFO

Keywords:
agentic AI
system identification
closed-form models
large language models
physical requirements
symbolic regression

ABSTRACT

System identification requires expert decisions on the identification method, incorporation of prior physical knowledge, and model validation. We propose ASPIRE, a framework for nonlinear system identification in which a large language model agent selects and executes the identification method. ASPIRE constructs closed-form, nonlinear static or dynamic models from measured data, optionally supplemented by a system description, the intended use of the model, and physical requirements expressed formally or in natural language. Each model includes its parameters, an estimation record, an interpretation of its terms, and an error evaluated on test data inaccessible to the agent. Formal physical requirements are verified independently. ASPIRE operates autonomously or interactively, allowing the agent to ask questions and the user to request further analyses and identification rounds. We evaluate ASPIRE on three measured systems: a mobile robot, a rotational system with dry friction, and a laboratory fermenter. Each system is tested with and without a description, in five runs per task. All 30 runs delivered valid models, at a cost of 0.96 to 3.38 US dollars per run. An assessment by the authors found that, for the 30 models recommended by the agent, one per run, the interpretation of the model and the documentation of its structure selection and parameter estimation fully satisfied 132 of 150 assessed items and partly satisfied 18. The most frequent discrepancy, found in nine models, concerned inaccurate or unsupported claims about model structure selection.

arXiv:2611.13114v1 [cs.RO] 9 Nov 2026

1. Introduction

In industrial applications, white-box models based on closed-form mathematical expressions are usually preferred to black-box models. Closed-form models relate inputs, states, and outputs through a small set of named parameters and, as such, are useful wherever a model must be interpreted, checked against physical knowledge, extrapolated beyond the range of the data, or embedded in model-based control. Obtaining such models from measured data requires several decisions by an expert: the identification method, the model class, how prior physical knowledge enters the estimation, the estimation criterion, and the validation procedure that matches the model's intended use. Techniques such as symbolic regression (SR) and sparse regression automate the search for model structure [3, 7], and some SR tools allow for incorporating physical requirements and prior knowledge [5, 6, 20, 23, 44].

Agents based on large language models (LLMs) that write and execute code can choose the identification method: they can inspect the data, propose model structures informed by physical knowledge, implement a fitting procedure, or call an existing SR package and iterate. Recent systems couple LLM-proposed equation structures with numerical parameter fitting [11, 36], give an LLM a fixed set of code-execution tools for equation discovery [43], or let an agent apply physics-based analysis tools to infer properties of the system and then configure existing SR and sparse-regression packages accordingly [48]. In system identification, ASIA [32] delegates the choice of the model class, the training algorithm, and the hyperparameters to an autonomous coding agent.

Existing agentic equation-discovery methods have several limitations: they do not support the specification and verification of physics-based requirements, evaluation on test data inaccessible to the agent, and documentation of the identification procedure. Furthermore, each delivered model should include a record of its structure and parameter estimation, distinguishing estimated parameters, fixed constants, and parameters derived from other parameters. The record should contain sufficient information to check the reported identification procedure against the executed code and execution logs. To the best of our knowledge, no published framework combines these features with agent-selected identification methods and a protocol for reporting variability across repeated runs.

We propose ASPIRE[1], a framework for system identification using LLM agents. ASPIRE constructs closed-form static or dynamic models from measured data, optionally supplemented by a system description, the model's intended use, and physical requirements expressed formally or in natural language. The agent selects and executes the identification method, which may involve physically motivated model structures, symbolic or sparse regression, or custom estimation code. Each candidate model is accompanied by its

*Corresponding author

erik.derner@cvut.cz (E. Derner); jiri.kubalik@cvut.cz (J. Kubalík); r.babuska@tudelft.nl (R. Babuška)

[1]ASPIRE stands for Agentic System-identification Platform for Iterative Reasoning and Exploration. It is unrelated to the robot-skill system of the same name [27].

parameter values, estimation procedure, interpretation, limitations, and test error. The agent also recommends one of the candidates. ASPIRE supports autonomous and interactive operation; in the latter, the agent can request information from the user, who can also request analyses and further identification rounds. The framework consists of one or more LLM agents responsible for identification and a software harness that executes their commands, enforces computational budgets, checks and evaluates candidate models, and records the identification runs.

In summary, the main contributions of this paper are:

- We introduce ASPIRE, an agentic framework for system identification in which LLM agents autonomously select, implement, and execute identification methods to obtain nonlinear closed-form models from measured data. Unlike systems based on predefined modeling pipelines or restricted toolboxes, ASPIRE allows the agent to develop and compare alternative model structures and estimation procedures, including methods not specified in advance.
- We integrate physical knowledge and user interaction into agentic system identification. Users can specify the intended application, physical requirements, and available knowledge about the system, while the agent can request clarifications, explain its results, and revise models based on user feedback. Formal physical requirements are checked independently of the agent, including properties such as monotonicity, equilibrium conditions, and physically consistent behavior under specified operating conditions, which cannot be assessed by predictive accuracy alone.
- We develop an identification framework that separates autonomous model construction from independent model verification and evaluation. The framework prevents agents from accessing test data, verifies formal physical requirements, and retains executable models, parameter estimation records, and execution logs. This separation enables the identification procedure and its results to be examined and reproduced, despite the agent's freedom to choose its methods.
- We demonstrate the framework on three experimentally measured systems: a mobile robot, a rotational system with dry friction, and a laboratory fermenter. Repeated experiments with and without system descriptions investigate the agent's ability to identify models from data alone and to incorporate prior physical knowledge. The results show that the agent can construct models with competitive predictive accuracy while satisfying physical requirements that are not enforced by conventional symbolic regression and sparse identification methods. The experiments also quantify run-to-run variability and computational cost.

The paper is organized as follows. Section 2 reviews related work. Section 3 defines the identification problem. Section 4 describes the ASPIRE framework. Section 5 describes the test systems, the compared methods, and the experimental design. Section 6 presents the results, and Section 7 discusses them. Section 8 concludes the paper.

2. Related Work

2.1. LLM-based and agentic equation discovery

A prominent early example of LLM-in-the-loop equation discovery is FunSearch [34], which combines LLM-generated programs with automated evaluation in an evolutionary loop. AlphaEvolve [30] extends this paradigm from evolving individual functions to evolving programs comprising multiple functions and components. LLM-SR [36] also builds on the FunSearch framework, adapting it to scientific equation discovery by combining LLM-proposed equation structures with numerical optimization of their constants. Several refinements retain this fixed-pipeline character [1, 10, 11, 28, 42], including hybrids that embed the LLM as a guided variation operator inside genetic-programming search over dynamical-system equations [51]. A complementary line uses LLMs to synthesize components of the evolutionary search itself, such as parent-selection operators [50]. Recent agentic approaches give LLMs control over the computational steps of equation discovery. SR-Scientist [43] provides an LLM with a fixed set of code-execution tools for data analysis and equation evaluation, which it uses to iteratively refine candidate equations. KeplerAgent [48] uses physics-based tools to infer properties of the system and configure symbolic-regression methods such as PySR and PySINDy. STRIDE [39] organizes equation discovery into modules with fixed roles and uses self-reflection to guide corrections. In system identification, ASIA [32] uses an LLM as an autonomous coding agent built on existing agentic platforms. Given a plain-language problem description, the agent chooses the model class, training algorithm, and hyperparameters. ASIA was tested on two system-identification benchmarks. Its authors identify implicit test leakage, reduced methodological transparency, and reduced reproducibility as limitations. The environment in which an agent operates is examined in [14], where production coding agents are treated as opaque model-discovery operators, and in a position paper on the design of agent environments [45]. Neither work addresses system identification with physical requirements and closed-form model outputs.

Table 1 compares ASPIRE with the three related systems. As in ASIA, the agent of ASPIRE chooses its identification method, and it is not restricted to a fixed set of methods as in SR-Scientist and KeplerAgent. ASPIRE differs from all three systems in the role of physical knowledge and in the output it delivers. The user states physical requirements either formally or in natural language, and the harness verifies the formal requirements for every delivered model. Every model is a closed-form expression with a record of its parameters, estimation procedure, and interpretation, and the execution logs allow this record to be checked against the agent's actions. Moreover, the evaluation protocol keeps the

Table 1
Comparison of ASPIRE with three related agentic identification and equation-discovery systems. The entries for ASIA, SR-Scientist, and KeplerAgent are based on the cited papers.

Property	ASPIRE	ASIA [32]	SR-Scientist [43]	KeplerAgent [48]
Identification method	chosen and implemented by the agent; no predefined method required	agent selects and implements model classes, training methods, and hyperparameters	agent writes analysis code and searches for equations using predefined tools	agent configures PySR and PySINDy using data analysis and inferred physical properties
Delivered model	closed-form expressions with parameter and estimation records	models from agent-selected classes, including neural and hybrid models	symbolic equation with fitted constants	symbolic equations, including differential equations
Physical knowledge of the user	descriptions and requirements in natural language; formal requirements verified	problem description, optional domain knowledge, and modeling suggestions	no documented support for user-specified physical requirements	physical properties inferred from data; no documented verification of user requirements
Test data	inaccessible to the agent; evaluated by the harness	held-out tests; potential implicit leakage acknowledged	separate test sets; agent isolation not specified	test error used for stopping and selection across runs

test data inaccessible to the agent, which addresses the test leakage named by the authors of ASIA.

2.2. Physical constraints and prior knowledge

Physical knowledge was used to constrain and guide symbolic regression before the introduction of LLM-based equation discovery. Existing approaches include multi-objective SR with constraint samples [20, 21] and search initialized with prior features [9]. Other methods add constraint losses into neural SR [19, 22]. These neural approaches have also been extended using asynchronous neuro-evolutionary search [23]. AI-Descartes and AI-Hilbert incorporate background axioms through logical reasoning and algebraic geometry [5, 6]. PG-SR encodes prior knowledge as executable constraint programs [44]. These methods use physical knowledge within predefined search procedures. ASPIRE supplies physical requirements to an agent that chooses its own identification procedure. The harness checks the formal requirements on each delivered model (Section 4.2).

2.3. Autonomous machine-learning and research agents

LLM-based agents also automate general machine-learning engineering. AIDE uses tree search over solution scripts [17]. Other examples include R&D-Agent [49], DS-Agent [13] and Kaggle-scale agents [12]. MLE-bench provides a benchmark for evaluating such agents [4]. Research-automation systems include the AI Scientist [26, 46], Curie [18], CodeScientist [16] and data-to-paper [15]. SWE-agent examines the interface between a language model and a software environment [47]. These systems deliver arbitrary models or artifacts, and none of them requires a closed-form model or verifies physical requirements. Agentic LLM systems have also been surveyed in robotics, where deployed systems are graded by their degree of autonomy, goal-directed behavior, adaptability, and decision-making [33].

2.4. Evaluation conventions

We report predictive accuracy and model complexity, as in SRBench [24]. For dynamic systems, the identified model is simulated on test trajectories using only the measured inputs [8, 35]. Shared SR infrastructure such as SR-Toolkit [29] standardizes pipeline components to prevent implementation details from confounding method comparisons. ASPIRE addresses the evaluation of agents, for which the evaluator must be sealed against the system under test. We do not run SR-Scientist, KeplerAgent or ASIA on the tasks of this paper. None of these systems accepts formal requirements, a specified evaluation mode and sealed test data as inputs; running them under the conditions of this paper would require adaptations of each system, and the results would then describe the adaptation rather than the published system. As reference points for the test error, we use two established tools for closed-form models with published default settings, PySINDy [3, 37] and PySR [7]. Like the agentic systems, these tools do not accept physical requirements; we therefore report their test scores and, separately, whether their models violate the formal requirements of the task (Section 5.5).

3. Problem Definition

We consider the identification of closed-form models of static and dynamic systems from measured data. The user specifies the available data, the required model form, and the evaluation procedure. Additional information, including a description of the system, its intended use, and physical requirements, may be provided, but is not mandatory. We first define the data and model representations supported by ASPIRE, followed by the assumptions and the identification objective.

The datasets are provided as tables with the columns *name*, *unit*, *description*, *roles*, and *training data range*. A *trajectory* is an ordered sequence of samples from one experiment, recorded with sampling period T_s. Each column may have one or more of the following roles: *input*, an exogenous signal read from the data during simulation; *state*, a component of the state vector computed by the model during simulation; *output*, a measured signal used for model evaluation; and *state derivative*, the sampled time derivative of a declared state, which may serve as a fitting target.

We denote the input vector by $u \in \mathbb{R}^{n_u}$, the state vector by $x \in \mathbb{R}^{n_x}$, and the measured output by $y \in \mathbb{R}$. The vectors p and c contain the parameters estimated from the data and the fixed constants, respectively. A hat denotes a model prediction or a parameter estimate.

Static models. A static model describes a memoryless relation between the current input and the output. The agent identifies an expression f for the predicted output,

$$\hat{y} = f(u; p, c), \tag{1}$$

where f is a scalar-valued function.

Dynamic models. A dynamic model describes the evolution of the system state. ASPIRE supports both continuous-time and discrete-time state-space representations. For continuous-time systems, the agent identifies the state-derivative function g_c,

$$\frac{\mathrm{d}x}{\mathrm{d}t} = g_c(x, u; p, c), \tag{2}$$

where t denotes continuous time. For discrete-time systems, the agent identifies the state-transition function g_d,

$$x(k+1) = g_d\big(x(k), u(k); p, c\big), \tag{3}$$

where k is the sampling instant. The functions g_c and g_d are vector-valued, with one expression for each state component. The measurement equation maps the state and input to the predicted output,

$$\hat{y} = h(x, u), \tag{4}$$

where h is given and may simply select one state component.

NARX models. A discrete-time task may also be formulated as a nonlinear autoregressive model with exogenous inputs (NARX) [25],

$$\hat{y}(k+1) = f_{\mathrm{N}}\big(y(k), \dots, y(k-n_a+1), u(k), \dots, u(k-n_b+1); p, c\big), \tag{5}$$

where f_{N} is the model expression, and n_a and n_b are the numbers of past outputs and inputs, respectively, specified by the task.

The NARX model is a special case of the discrete-time state-space model (3) with measurement equation (4). Its state vector is $x(k) = [y(k), \dots, y(k-n_a+1)]^\top$, with components $x_1, \dots, x_{n_a}$. The first component of g_d is the unknown function f_{N}, whereas the remaining components are defined by the known shift equations $x_i(k+1) = x_{i-1}(k)$, $i = 2, \dots, n_a$. Thus, f_{N} is the only expression to be identified, and h selects the first state component x_1. The past inputs $u(k-1), \dots, u(k-n_b+1)$ are read from the data in the same way as the current input $u(k)$.

The distinction between prediction and simulation is relevant. In one-step-ahead prediction, the past outputs in (5) are taken from the measurements. In simulation, the model uses its own outputs computed at preceding steps. The same distinction analogously holds for state-space models.

Assumptions and identification objective. The framework considered in this paper is subject to the following assumptions:

(A1) The output y by which a model is scored is a scalar; the state vector may have several components, all of which are identified and simulated.

(A2) All states are measured or can be constructed directly from measured signals, so that no state estimation is required.

(A3) The measurement equation (4) is given and is not estimated.

The identification task is to determine the expression f in (1), g_c in (2), or g_d in (3), including the NARX representation (5). The identified model must be accurate on test data under the specified evaluation mode (Section 4.5), satisfy the prescribed physical requirements, and be suitable for its intended use. The identified expressions must be closed-form, consisting of finite combinations of the variables, estimated parameters, fixed constants, and permitted mathematical functions. For dynamic systems, this restriction applies to the state-derivative or state-update equations, whereas computing the resulting trajectories may require numerical integration.

4. The ASPIRE Framework

4.1. Overview

ASPIRE consists of a *harness* and one or more *agents* (Figure 1). The agent is a language model that carries out the identification: it inspects the data, proposes model structures, writes and runs estimation code, and explains its models. The harness is the software around the agent: it passes the task to the agent, executes the agent's commands in a working directory, enforces a budget, checks

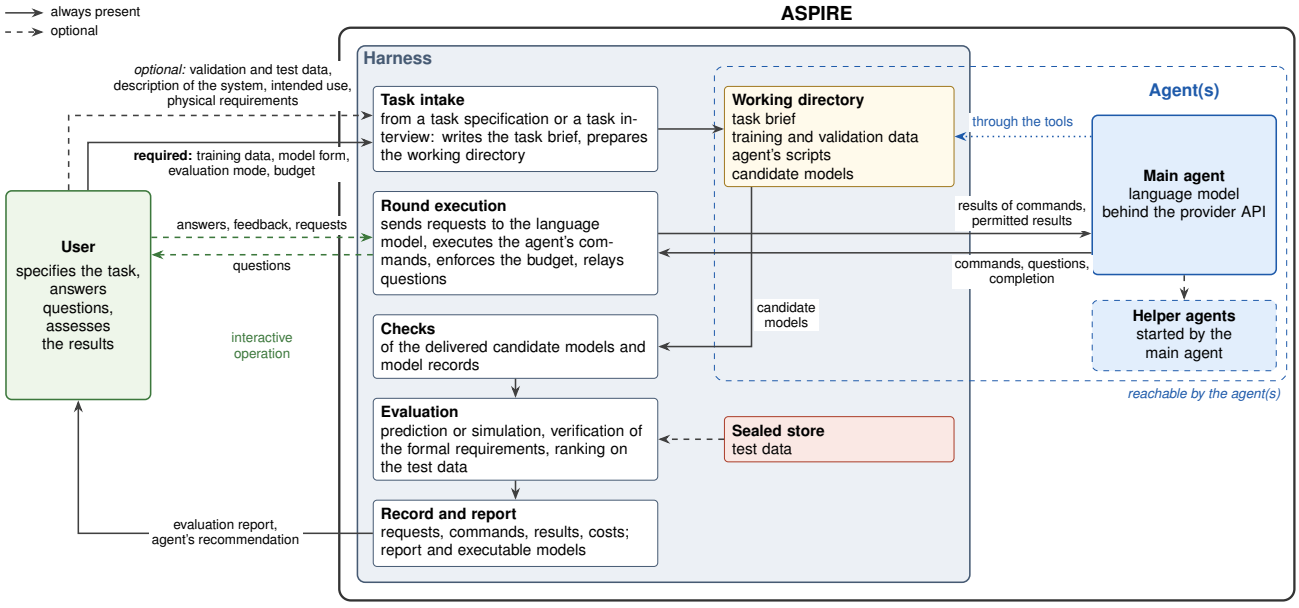


Figure 1: Overview of ASPIRE, which consists of the harness (gray) and the agent(s) (blue). Solid arrows and boxes are always present. Dashed ones are optional: the optional part of the task specification, the interactive operation (green) and the helper agents. The agent acts on the working directory only through the tools that the harness executes (dashed blue boundary) and never reaches the sealed test data.

and evaluates the delivered models, and stores a complete record of the session. The *user* specifies the task, answers the agent's questions in interactive operation, and assesses the results. The test data stay with the harness and are *sealed*, that is, inaccessible through the agent's files, tools, and conversation.

One attempt by the agent to produce or revise a set of models within the budget is a *round*, and the set of models delivered at the end of that round is the *pool*. A *session* consists of one round in autonomous operation, and of several rounds with analyses between them in interactive operation (Section 4.6). After each round, the user receives the models with their errors on the training, validation, and test data, the outcomes of the physical requirements, the agent's recommendation, and the models as executable code. The harness presents these results as a report for reading in a browser (Section 6.3).

4.2. Task specification

A task has a compulsory and an optional part. The user must provide the training data, the model form of Section 3 with the roles of the data columns, the evaluation mode (Section 4.5) and the budget; a dynamic task also needs the sampling period and the measurement equation (4). Everything else is optional: validation data, test data, a description of the system and of the data, the intended use of the model, physical requirements, known constants, and references to model families from the literature. The division of the data into training, validation, and test data is made by the user. The agent receives this information as a *task brief*, a text that the harness generates from the specification by fixed rules, without a language model, and that also states what the user did not provide.

By default, a model may contain the arithmetic operations, powers, the elementary functions (exp, log, the square root, the trigonometric and hyperbolic functions and their inverses), the absolute value, the sign, the minimum and the maximum, and piecewise definitions with relational and logical conditions; the user may adapt this set. Every value obtained by parameter estimation is a named parameter, whereas a numeric literal that belongs to the form of an expression, such as the exponent in x^2, is not.

The user states physical knowledge either as formal requirements or in natural language. A formal requirement has one of the nine types of Table 2, which consolidates the constraint types of the physics-aware SR methods of [20–22], and it specifies the expression concerned, the domain, and the tolerance. For example, the states of the friction system of Section 5.2 are an angle θ and an angular velocity ω, and a requirement of the type equation, $\mathrm{d}\theta/\mathrm{d}t - \omega = 0$, ensures that every delivered model keeps this kinematic relation. The harness verifies the formal requirements on every delivered model (Section 4.5), and by default, a demonstrated violation excludes the model from the ranking and from the recommendation. Requirements in natural language are passed to the agent verbatim and are assessed by the user. The user either states the requirements in natural language or enters necessary conditions of the nine types that follow from it, each on a stated domain; Table 4 gives an example of formal requirements, and Appendix C gives an example of requirements in natural language.

Table 2
Types of formal physical requirements.

Type	Statement
Symmetry	invariance under a permutation of variables
Monotonicity	sign of the partial derivative with respect to one variable on a domain
Parity	odd or even under a sign change of one or several variables jointly
Exact value	value at a specified point, with a tolerance
Inequality	relation between the output and the variables on a domain
Curvature	convexity or concavity with respect to one variable
Limit	value as a variable tends to a limit
Equation	an expression that must equal zero, possibly at a point
Relation at points	a relation required at a specified set of samples

4.3. Candidate models and the model record

The agent is instructed to explore different identification approaches, such as physically motivated modeling, symbolic regression (SR), and sparse regression, and to deliver a *pool* of candidate models with distinct structures rather than variations of the same model form. The maximum number of candidates is specified by the user. The agent may recommend one candidate based on the training and validation data, and this recommendation is recorded before any test results become available.

Each *candidate* consists of the model equations and a *model record* with three parts: a parameter table, an estimation record, and a discussion.

The *parameter table* lists every named quantity in the equations, specifying its value, unit, type, and description. Three parameter types are distinguished: *fitted*, for values estimated from data; *fixed*, for values specified independently of estimation, with their source identified (the task, physics, literature, normalization, or a modeling choice); and *tied*, for values computed from other parameters using a stated equation.

The *estimation record* specifies the estimation criterion, the data used, the estimation method, and its starting values, together with the reasons for these choices. It also identifies the script that reproduces the estimated values.

The *discussion* identifies the model family, interprets each term in the equations, explains how the model structure was obtained, and describes the model's limitations. Terms without a known physical interpretation are characterized by their mathematical effects. The account of model construction distinguishes structures derived from physical reasoning, obtained through search, or proposed as hypotheses, and documents the alternatives considered and the reasons for subsequent modifications.

The model record follows a common format across all tasks. The agent receives a template with an example based on an artificial system, along with instructions to define every symbol, specify units for numerical values, cite only verifiable references, justify modeling choices, and avoid unsupported physical interpretations. Section 6.3 summarizes a delivered record, and Appendix B gives three delivered models with their parameter tables.

The harness checks the completeness and consistency of each record with the model equations. In particular, all symbols must be declared, all parameters must be used, and the expressions must be real and finite on the training data. These checks do not establish whether the reported estimation procedure was actually followed or whether a parameter declared as fitted was estimated as stated. The harness, therefore, retains the agent's scripts and execution logs, allowing the reported identification procedure to be checked against the actual computations.

4.4. Agent and budget

The agent may read and write files in its working directory, write and run code, search the web, install software packages, and delegate subtasks to helper agents, which are further instances of the language model. Its environment contains Python with numerical libraries, among them SciPy [41], the SR package gplearn [38], and the sparse-regression package PySINDy [37]; whether it uses them is its decision. Models of any kind, including black-box models, may serve as intermediate tools, but only closed-form models are to be delivered. The instruction that the agent receives with every request – the system prompt – is given in Appendix A. No tool gives access to the test data or to evaluation results. Restricting the agent, for example, to forbid web access, can be stated in the task description.

An identification round is summarized in Algorithm 1. A *turn* is one response of the language model, which either requests actions or declares the pool complete. A round is limited in the number of turns, in the execution time, and in the cost of the language model; the default limits are 60 turns, 3600 s, and 10 US dollars. When the agent declares the pool complete, the harness checks the candidates; if none pass, the agent receives the reasons and may correct the pool once. When a limit is reached, the harness evaluates all candidates delivered so far to ensure valid models are not discarded. The harness records every request, response, command, and result of a session, along with the language model identifier, software versions, and cost.

4.5. Evaluation

The harness computes all predictions used in the evaluation, with the same settings for all candidates. A static model is evaluated algebraically. A dynamic model is evaluated either by one-step-ahead prediction from the measured state or by simulation, as specified in the task. In simulation, the state is initialized once per trajectory, and the model state is propagated using the recorded inputs without correction from later measurements. By default, a continuous-time model is integrated by the fourth-order Runge–Kutta

Algorithm 1 One identification round. The harness executes the steps; the agent produces the responses.

give the agent the task brief and the training and validation data
while no budget limit is reached **do**
    obtain the next response of the agent ▷ one turn
    if the response requests actions **then**
        execute them in the working directory and return the results
    else if the response declares the pool complete **then**
        check the candidates
        if a candidate passes or a correction was already requested **then**
            break
        end if
        return the reasons for the rejection
    end if
end while
record the recommendation; evaluate every complete candidate; verify the formal requirements

method, with one step per sampling period and the input held constant over the step.

A simulation error grows with the simulated length, so that simulation errors on trajectories of different lengths are not comparable. If the test trajectories are shorter than the training and validation trajectories, the user may therefore specify a horizon H, the number of sampling periods simulated in a test trajectory. The harness then divides every training and validation trajectory into consecutive segments of H sampling periods that do not overlap; the last segment of a trajectory may be shorter. Each segment is simulated on its own: the state is initialized from the measured state at the first sample of the segment and is propagated to the end of the segment without correction. The starting points of the simulations are thus H samples apart. This differs from a receding horizon, in which a simulation of H samples would start at every sample. Every sample after the initialization of a trajectory is scored exactly once. The training and validation errors are the root mean squared errors (RMSEs) pooled over all segments, computed using (6) with the segments in place of the test trajectories. The test trajectories are not divided.

The candidates are ranked by the RMSE of the output, pooled over the test trajectories $j = 1, \dots, J$,

$$S = \sqrt{\frac{\sum_{j=1}^{J} \sum_{k \in I_j} \left(\hat{y}_j[k] - y_j[k]\right)^2}{\sum_{j=1}^{J} N_j}}, \tag{6}$$

where $y_j[k]$ is the measured output, $\hat{y}_j[k]$ the predicted or simulated output, I_j the set of scored samples of trajectory j and N_j its size. The test score S has the unit of the output. A candidate is *eligible* for the ranking if it passes the checks of Section 4.3, has no demonstrated violation of a formal requirement, and completes the evaluation on every test trajectory with real and finite values.

The *recommendation* of the agent is made without test data. In contrast, the ranking by S uses the test data: it shows whether the pool contains an accurate model, not whether that model can be recognized without test data. We therefore call the lowest S in a pool an *oracle-based selection* and always report it with the pool size.

The harness verifies each formal requirement symbolically where its type permits, and otherwise by sampling its domain. The outcome is a proof, a counterexample or an inconclusive result; a sampled check that finds no violation is inconclusive.

Because language model responses are stochastic, a task is commonly run several times under identical settings, called an *experiment*. For an experiment, we report the number of runs attempted, the number of runs that delivered a pool passing the checks, and the number of runs with at least one eligible candidate. Each quantity is reported as the mean and the sample standard deviation across the runs, along with the cost.

4.6. Interactive operation

In interactive operation, the user takes an active part throughout the session. During a round, the agent may ask the user a specific question whose answer would affect a modeling choice. If the user cannot answer, the agent states the assumption it makes and how that assumption limits the result. After a round, the user receives the complete evaluation, including the test errors, whereas the agent receives only the results on the training and validation data. The user may then ask the agent for an explanation or a further analysis, or give feedback and request a new round, in which the agent revises the pool; earlier pools are kept. Because the user may have seen test results before giving feedback, every later round is labeled *test-informed* and is not presented as an independent test of the identification.

A session can also start from the user's data files alone, without any accompanying description or metadata. In a *task interview*, an agent examines the training and validation data, proposes what the data show (the roles of the columns, the sampling period, the ranges), asks the user for what the data do not determine (what the system is, what the model will be used for, the units, the physical requirements), and writes the task specification. The user confirms or edits the resulting task brief before the identification starts. The test data are sealed from the interview and from every agent. We use a language model, not a fixed questionnaire, for the interview, so that a term can be explained to the user and an unclear answer can be followed up on.

5. Experimental Setup

We evaluated ASPIRE on three systems with measured data: a mobile robot, a rotational motion under dry friction, and the pressure dynamics of a fermenter (Table 3). All

Table 3
The three test systems. Model form: discrete-time (3) or continuous-time (2). Data: number of trajectories and of samples; for the test data, the number of scored samples, which excludes the first sample of each trajectory.

	Mobile robot	Friction system	Fermenter
Model form	discrete-time	continuous-time	continuous-time
States	position x, y (m), heading ϕ (rad)	angle θ (rad), angular velocity ω (rad/s)	pressure P (bar)
Inputs	forward velocity v_f (m/s), angular velocity v_a (rad/s)	motor voltage u (V)	valve position u_v (% closed), air flow rate u_f (l/min)
Output	x	θ	P
Sampling period T_s	0.2 s	0.01 s	5 s
Training data	3 trajectories, 450 samples	13 trajectories, 3522 samples	1 trajectory, 1452 samples
Validation data	none	3 trajectories, 1192 samples	1 trajectory, 1454 samples
Test data	12 trajectories, 298 scored samples	4 trajectories, 978 scored samples	1 trajectory, 1209 scored samples
Horizon H	25 samples	none	none
Formal requirements	none	R1–R3	none
Requirements in natural language	none	none	R1–R5

three tasks are dynamic, and the models are evaluated by simulation.[2]

5.1. Mobile robot

The experimental platform is a TurtleBot 2 mobile robot with a differential drive, which moves on a flat floor. The state is the pose of the robot logged by the on-board odometry: the position coordinates x and y in meters and the heading ϕ in radians, where $\phi = 0$ is the direction of the x axis and the logged heading is wrapped to the interval $(-\pi, \pi]$. The input consists of the two commands sent to the robot: the forward velocity v_f in m/s and the angular velocity v_a in rad/s. For this system, x and y denote the two position coordinates and not the state vector and the output of Section 3: in the notation of that section, the state vector is $[x, y, \phi]^\top$, the input vector is $[v_f, v_a]^\top$, and the measured output is the position x. The data columns have the names `x`, `y`, `phi`, `v_f`, and `v_a`. The agent provides the three components of g_d in (3), one expression for the next value of each state. The commands were drawn at random and held for five samples each. The logged pose carries a considerable measurement error of unknown magnitude, and the heading responds to the angular velocity command with a delay of about one sample, which a model of this form cannot represent.

The data are five trajectories of 150 samples each, recorded with a sampling period $T_s = 0.2$ s. Three trajectories are the training data. The other two are split into 12 test trajectories of 25 samples (5 s), in accordance with the intended use stated in the task: simulating the position x over about 5 s from one logged pose and the sequence of commands. No physical requirements are stated.

[2]The harness, the task specifications with their data, and the complete records of all runs are available in a public repository: `https://github.com/erik-derner/aspire-sysid`.

5.2. Rotational motion under dry friction

The second system is a laboratory setup [23] in which a voltage-controlled direct-current motor drives a vertical disc with an off-center weight, so that gravity acts on the rotation. A leaf spring presses a felt pad against the disc, adding dry friction to the bearings. We refer to it as the friction system. The data contain four signals. The state vector is $[\theta, \omega]^\top$, where θ is the angle of the disc in radians (column `theta`), measured by an encoder, with $\theta = 0$ at the stable equilibrium, and ω is the angular velocity in rad/s (column `omega`). The input is the motor voltage u in volts (column `u`), a scalar. The fourth signal is the angular acceleration $\mathrm{d}\omega/\mathrm{d}t$ in rad/s^2 (column `domega`); it serves as a state derivative and is provided as a possible fitting target. The angular velocity and the angular acceleration are obtained from the measured angle by numerical differentiation. The measured output is the angle θ. The agent provides the two components of g_c in (2), the expressions for $\mathrm{d}\theta/\mathrm{d}t$ and $\mathrm{d}\omega/\mathrm{d}t$. The data consist of driven trajectories, which start at rest under a random binary voltage, and of free decays, which start from an initial angle with $u = 0$ and end with the disc sticking at rest. The description given to the agent names the two sources of friction and the two kinds of trajectories. The intended use is to simulate θ over a full trajectory, with stable behavior over a long horizon and realistic stick-slip behavior.

The task states three physical requirements, all of them formal (Table 4). We denote by $f(\theta, \omega, u)$ the expression for $\mathrm{d}\omega/\mathrm{d}t$. R1 is the kinematic relation $\mathrm{d}\theta/\mathrm{d}t = \omega$. R2 states that gravity acts toward the stable equilibrium, whereas static friction may prevent motion: at rest and without voltage, the angular acceleration is zero or points toward $\theta = 0$, that is, $f(\theta, 0, 0)\,\theta \le 0$ for every angle. R3 states the influence of the control input: f does not decrease as the voltage increases; it may remain constant, as it does when the system sticks due to friction. The tolerance of every requirement is zero, and a demonstrated violation excludes a model. The brief gives the agent the three conditions with their domains, as in Table 4, without a verbal statement.

Table 4
Formal requirements of the described task of the friction system. $f(\theta, \omega, u)$ denotes the delivered expression for $\mathrm{d}\omega/\mathrm{d}t$ in rad/s^2, with θ in rad, ω in rad/s and u in V. Types as in Table 2. The tolerance of every requirement is zero.

Requirement	Type	Condition	Domain
R1	equation	$\mathrm{d}\theta/\mathrm{d}t = \omega$	$\theta \in [-3.1416, 3.1416]$, $\omega \in [-23, 23]$, $u \in [-10, 10]$
R2	inequality	$f\ \theta \leq 0$	$\theta \in [-\pi, \pi]$, $\omega = 0$, $u = 0$
R3	monotonicity	f non-decreasing in u	$\theta \in [-3.1416, 3.1416]$, $\omega \in [-23, 23]$, $u \in [-10, 10]$

5.3. Pressure dynamics of a fermenter

A 40-liter laboratory fermenter is filled with about 25 liters of water [2, 20, 40]. Air is fed from the bottom at a flow rate set by a mass-flow controller and exits through an outlet valve at the top, so the pressure in the headspace depends on the airflow rate and the valve position. The data contain four signals. The only state is the air pressure in the headspace in bar (column `p`), which is also the measured output; we denote it by P to keep p for the parameter vector. The input consists of two signals: the position of the outlet valve u_v in percent closed (column `valve`), where 100 % is fully closed, and the air flow rate u_f in l/min (column `flow`). The fourth signal is the time derivative $\mathrm{d}P/\mathrm{d}t$ in bar/s (column `dp`), computed from the pressure using a filtered numerical difference; it serves as a state derivative and is provided as a possible fitting target. The pressure is quantized. The agent provides g_c in (2), which here is the single expression for $\mathrm{d}P/\mathrm{d}t$. The training, validation, and test data are each a single trajectory, so the test score is based on a single trajectory. The intended use is the simulation of the pressure over a whole trajectory, with stable behavior over a long horizon and with a model whose structure can be defended from the physics and not only from the fit.

The task states five physical requirements in natural language: at a fixed pressure, the rate of the pressure does not decrease when the air flow rate increases (R1) or when the valve is closed further (R2); with zero air flow and an open valve, atmospheric pressure is the equilibrium (R3) and the pressure tends toward it (R4); and with a fully closed valve the pressure cannot decrease while air is supplied (R5). See Appendix C for full detail. The agent receives the requirements verbatim. The harness does not verify them; we assessed them on the recommended models as described in Section 5.6. No formal requirements are stated.

5.4. Described and blind tasks

Every system is run as two tasks. The *described* task contains the information given above. The *blind* task contains the same data (the same dataset split), sampling period, and column roles, and the same evaluation settings, but nothing that tells what the system is: the columns are named $x_1, x_2, \ldots$ and y, and the units, the description, the intended use, the ranges, and the requirements are omitted.

This allows us to assess the effect of prior knowledge on the identified models. The blind tasks also reduce the possibility that the agent reproduces a known physical model from its training data rather than inferring the model structure from the measurements. Finally, they represent system identification with limited prior knowledge, a common setting in symbolic regression.

5.5. Compared methods

We compare the models of the agent with those of two established tools for closed-form models, applied to the training data of each system:

- PySINDy [37] – sparse identification of nonlinear dynamics [3] with a polynomial library of degree 2 in the states and inputs and sequentially thresholded least squares;
- PySR [7] – SR by regularized evolution of expression trees.

Each tool is used at two settings. The *default* setting is that of the package: the threshold 0.1 for PySINDy, and for PySR, the four arithmetic operators and the equation that the package selects. The *matched* setting brings the tool closer to the agent's conditions. For PySINDy, the threshold is selected from a grid. For PySR, the unary functions sin, cos, tanh, exp, log, the square root, the absolute value, and the sign are added, and the equation is selected from the Pareto front of accuracy and complexity. Both selections minimize the simulation error on the training data. The tools fit the next value of each state for the mobile robot and the supplied derivative for the other two systems. The harness evaluates their models exactly as the agent's candidates, including verification of the formal requirements. The tools do not receive the requirements or system description. We therefore report the test score for each model and, separately, the number of its models with a demonstrated violation, which would exclude a candidate of the agent. PySINDy is deterministic; PySR is run with five seeds, and the results are reported as the mean and sample standard deviation.

5.6. Assessment of the model records

The harness checks the form of a model record; it does not check its content (Section 4.3). We therefore assessed the record of the recommended candidate of every run, 30 records in total, by five items, each marked *yes*, *partly* or *no*:

(a) every term of the expressions is interpreted, and the interpretation is correct; for a blind task, a term is described by its mathematical effect, and no physical meaning is invented;

(b) the stated origin of the structure agrees with the commands and results in the record of the run;

(c) the declared type of every parameter (fitted, fixed, or tied) agrees with the estimation script;

(d) the criterion, the estimation method, and the starting values each have a stated reason;

(e) the stated limitations follow from the data or from the structure of the model.

An item is marked *yes* when everything that it concerns satisfies it, *partly* when one term, claim, parameter, reason, or limitation does not, and *no* when several do not. The assessment was performed initially by Claude Code using Opus 5.5 on high effort on 8 October 2026, and manually verified by the authors.

The requirements in natural language for the fermenter were assessed using the recommended model for each of the five runs of the described task, with two marks per requirement. The first mark states whether the discussion in the model record addresses the requirement (*yes* or *no*). The second states whether the delivered expression with the delivered parameter values satisfies the requirement (*yes*, *partly*, *no*, or *not decidable* from the expression). It rests on evaluating the expression on a grid of valve positions from 0 to 100 %, air flow rates from 0 to 31 l/min, and pressures from 0.9 to 2.3 bar, ranges that contain the training data. R1, R2, and R5 are evaluated at pressures of at least 1 bar, because below atmospheric pressure, air enters through the outlet valve and the statements do not apply. For R3 and R4, a valve position of at most 90 % counts as open, and an equilibrium pressure within 0.05 bar of 1 bar counts as atmospheric. A requirement is marked *partly* when it holds on a part of the grid: for R1, R2, and R5 on at least 95 % of the grid points, for R3 and R4 at some of the open valve positions.

5.7. Settings

All runs used the language model `gpt-6.1-sol` of OpenAI [31] with medium reasoning effort in the role of the agent. The provider has not published a dated snapshot of this model; the runs are dated from 7 to 9 October 2026. Each of the six tasks was run five times, which gives six experiments and 30 runs. Every run was one round in autonomous operation with the default budget of Section 4.4 and a pool of at most 10 candidates. The cost is computed from the provider's prices at the time of the runs: 2 and 10 US dollars per million input and output tokens, respectively.

6. Results

All 30 runs ended when the agent declared its pool complete, and no run reached a budget limit; the largest consumption of a run was 52 turns, 2242 s and 3.38 US dollars. Every run delivered a pool that passed the checks, and all 126 candidates of the 30 pools are eligible: no candidate was excluded by a formal requirement, and no simulation failed. In every experiment, five runs were therefore attempted, five delivered a pool, and five had an eligible candidate. The agent recommended a candidate in every run. A summary of the six experiments can be found in Table 5.

After every run, the harness scanned the agent's files and conversation for values of the sealed test data. It reported 16 findings across 7 runs, and we examined each. All are numbers that the agent computed from the training or validation data, or copied from them into its own files, and that coincide with a test value in at least six significant digits. No finding indicates access breach to the test data.

6.1. Comparison with PySINDy and PySR

Table 6 shows the test scores of the models recommended by ASPIRE with those of the models found by PySINDy and PySR.

Mobile robot. The test scores of the ten recommended models lie between 0.0379 and 0.0443 m, below those of all five PySR models at the default setting (0.0458 to 0.0612 m). In the matched setting, PySR has a lower training error than in the default setting (0.0263 vs. 0.0654 m) and a higher test score; one of its five models (0.0438 m) lies within the range of the agent's models. This indicates that selection based on simulation error across three training trajectories favors equations that do not generalize to the test data. PySINDy at the default threshold removes every term that contains a command, so that its model predicts almost no motion (0.2140 m); with the selected threshold, it keeps 55 polynomial terms (0.1530 m).

Friction system. The test scores of the five recommended models of the described task lie between 0.0292 and 0.0459 rad, and those of the blind task between 0.0322 and 0.0492 rad. PySR at the matched setting (Section 5.5), in which the unary functions are added to the arithmetic operators, finds a sine of the angle and a hyperbolic tangent of the angular velocity and reaches 0.0363 ± 0.0039 rad (0.0335 to 0.0431 rad over the five seeds). Its mean lies 0.0054 rad below that of the described task and 0.0045 rad below that of the blind task. Both differences are smaller than the sample standard deviation of the agent's runs (0.0070 and 0.0079 rad), and we observe no difference between the three. The models differ in another respect. All 12 models of the two tools violate the requirement R2 of the described task: each contains a constant term or a polynomial in the angle, so that at rest and without voltage, its acceleration points away from the equilibrium at some angles. For the PySR models in the matched setting, the violation is small: for the counterexamples the harness found, at angles of 0.05 to 0.10 rad, the product $f\theta$ lies between 0.008 and 0.071 $\mathrm{rad}^2/\mathrm{s}^2$. Each of the 12 models would nevertheless be excluded as a candidate of the agent, whereas none of the 22 candidates of the agent in the described task has a demonstrated violation (Section 6.2). The other three models cannot express the gravity torque: PySR at the default setting has only the four arithmetic operators, and PySINDy uses a polynomial library of degree 2 at both settings, because its matched setting selects only the threshold. These models reach 0.144 to 0.269 rad.

Fermenter. The largest test score of the ten recommended models is 0.0239 bar, below the score of every model of the two tools. The best of these is the PySINDy model with

Table 5
Results of ASPIRE per experiment: mean ± sample standard deviation over five runs (five finite values in every entry). Training error: RMSE of the simulated output of the recommended model on the training data, for the mobile robot in segments of $H = 25$ samples. S: test score (6). Recommended: the model that the agent recommended without test data. Lowest in pool: oracle-based selection on the test data, with the pool sizes of the five runs. Errors in m (mobile robot), rad (friction system) and bar (fermenter). Language model `gpt-6.1-sol`, runs of 7 to 9 October 2026.

System	Task	Training error, recommended	S, recommended	S, lowest in pool	Pool sizes	Fitted param., recommended	Cost per run (US dollars)
Mobile robot	described	0.0397 ± 0.0007	0.0430 ± 0.0010	0.0382 ± 0.0016	6, 4, 6, 5, 4	2.4 ± 0.9	1.31 ± 0.25
Mobile robot	blind	0.0394 ± 0.0014	0.0401 ± 0.0015	0.0365 ± 0.0019	3, 3, 4, 4, 6	9.6 ± 5.9	1.91 ± 0.47
Friction system	described	0.0511 ± 0.0085	0.0417 ± 0.0070	0.0279 ± 0.0075	6, 3, 6, 4, 3	7.6 ± 1.7	2.20 ± 0.50
Friction system	blind	0.0459 ± 0.0096	0.0408 ± 0.0079	0.0387 ± 0.0067	3, 3, 3, 4, 3	8.4 ± 1.5	2.09 ± 0.45
Fermenter	described	0.0253 ± 0.0008	0.0232 ± 0.0006	0.0231 ± 0.0005	4, 3, 4, 5, 3	5.6 ± 1.1	2.01 ± 0.45
Fermenter	blind	0.0250 ± 0.0024	0.0225 ± 0.0010	0.0223 ± 0.0007	4, 6, 5, 5, 4	19.0 ± 2.2	2.70 ± 0.44

Table 6
Test score S (6) and number of fitted parameters of the models recommended by the agent and of the models found by PySINDy and PySR (Section 5.5), evaluated by the harness on the same test data. ASPIRE: mean ± sample standard deviation over five runs; PySR: over five seeds, with the range of the number of parameters; PySINDy: a single deterministic model. The lowest mean S of each system is set in bold; no statistical test was applied. †: every model of the tool violates the formal requirement R2 of the described task of the friction system (Table 4); a candidate of the agent with such a violation is excluded.

	Mobile robot		Friction system		Fermenter	
Method	Param.	S (m)	Param.	S (rad)	Param.	S (bar)
ASPIRE, described task	2.4 ± 0.9	0.0430 ± 0.0010	7.6 ± 1.7	0.0417 ± 0.0070	5.6 ± 1.1	0.0232 ± 0.0006
ASPIRE, blind task	9.6 ± 5.9	**0.0401** ± 0.0015	8.4 ± 1.5	0.0408 ± 0.0079	19.0 ± 2.2	**0.0225** ± 0.0010
PySINDy, default	3	0.2140	8	0.2668†	0	0.2631
PySINDy, matched	55	0.1530	10	0.2692†	10	0.0284
PySR, default	6–8	0.0504 ± 0.0068	4	0.1440 ± 0.0010†	3–5	0.0418 ± 0.0054
PySR, matched	8–14	0.0558 ± 0.0129	5–6	**0.0363** ± 0.0039†	1–5	0.0748 ± 0.0632

the selected threshold, which keeps all ten polynomial terms (0.0284 bar). PySR at the matched setting varies between 0.0276 and 0.1850 bar across the five seeds: equations with a small error in the derivative drift or leave the data range in the simulation.

Figures 2 and 3 show the measured and simulated outputs on the test data for one recommended model per described task. The run shown is the one whose recommended model has the median test score of the five runs; we fixed this rule before inspecting the results.

6.2. Delivered models

Among the resulting pools in all experiments, each pool contains 3 to 6 candidates, and within each pool, each candidate has its own structure.

Formal requirements are stated only in the described task of the friction system. The harness verified the three requirements in Table 4 for each of the 22 candidates across the five runs. No verification demonstrated a violation, so no candidate was excluded. R1 was proved for all 22 candidates. For R2 and R3, the symbolic stage reached a proof for 5 and for 3 candidates. For the other candidates, whose expressions contain minima and maxima, the evaluation at 10000 sampled points found no violation, which the harness reports as unresolved and not as satisfied. In total, 30 of the 66 verifications ended with a proof and 36 without a decision. In all five runs, the agent checked R2 and R3 itself before it declared the pool complete, by evaluating the fitted expressions at 1000 to 100000 sampled points of the stated domains; no check led to a change of a candidate. Four of the five recommended model records also derive the two requirements from the expression (Appendix B).

In the blind task of this system, no requirement is stated; all 16 candidates nevertheless give the rate of the output as the second state. To see what the requirements exclude, we applied the requirements of the described task to the five recommended models of the blind task, after renaming their variables. All five satisfy R1 and R3 and violate R2: each contains a constant term or a shift of the angle, so that at rest and without input, its acceleration is not zero at the equilibrium and points away from it on one side.

No run executed gplearn or PySINDy, the two packages for the search of closed-form models that were installed in the agent's environment (Section 4.4) and named in the system prompt (Appendix A). The agent fitted structures that it wrote itself, in several runs by least-squares regression on a library of candidate terms of its own choice. It started a helper agent in 27 of the 30 runs, searched the web in 27 runs and installed a package, a compiler for numerical Python code, in 14 runs.

With the description, the recommended models are physical structures in all 15 runs. For the mobile robot, they are the kinematic model of a unicycle with one gain per command. For the friction system, they combine a gravity torque proportional to the sine of the angle, in three runs

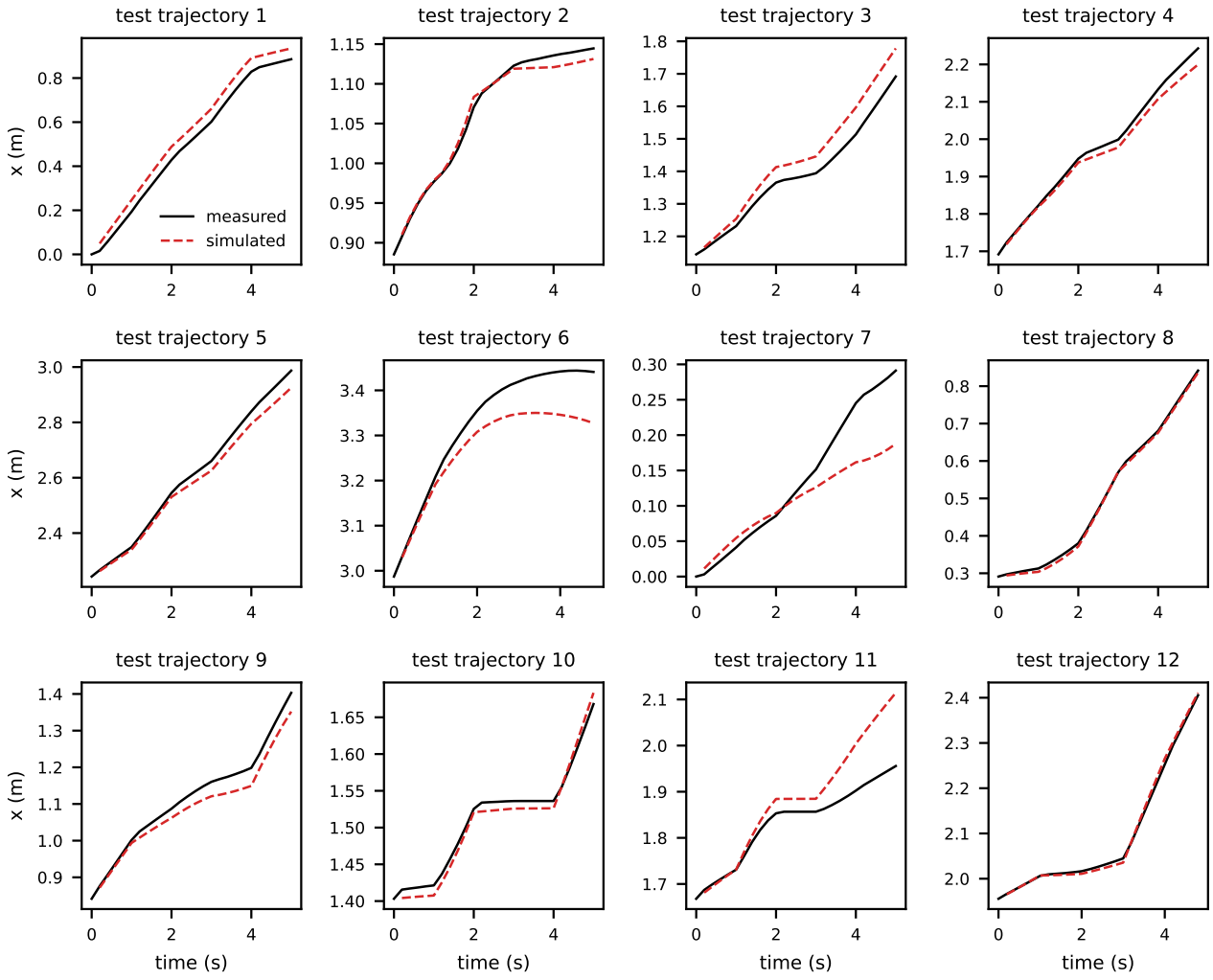


Figure 2: Mobile robot, described task: measured (solid) and simulated (dashed) position x on the 12 test trajectories for the recommended model of the run with the median test score ($S = 0.0428$ m). Each simulation starts from the measured pose and uses the recorded commands without correction.

with a second harmonic, viscous friction, Coulomb friction, and the voltage drive with a mechanism for sticking at rest. No requirement demands such a mechanism; the description and the intended use name it. In three runs, it is a term that cancels the applied torque at rest up to the friction level and whose weight, a smooth function of the angular velocity, decays over 0.12 to 0.16 rad/s (Appendix B). In the other two runs, the friction term is the applied torque plus a term proportional to the angular velocity, limited to the friction levels, which drives the velocity to zero while the applied torque lies within these levels. Two of the five models have a friction level of its own for each direction of motion, two a friction level that depends on the angle, and one a saturating characteristic of the motor. All five expressions are continuous in the angular velocity. For the fermenter, they are balances of the gas in the headspace with an inflow proportional to the air flow rate, an outflow through a nonlinear resistance and a monotone empirical valve characteristic. Without the description, the agent arrived at the same kind of structure for the mobile robot in three and for the friction system in four of five runs; for the friction system, these models contain, in the neutral names, a sine of the output, a linear term and a smoothed sign of the second state, and a gain of the input. In the other blind runs of these two systems, the recommended models are regression models on trigonometric and polynomial terms (mobile robot, two runs) or replace the sine by a polynomial of degree five (friction system, one run). For the fermenter, all five blind models are third-degree polynomials in the three variables.

6.3. Model records

ASPIRE presents the results of every experiment and of every interactive session as a report, a set of HTML pages that are read in a browser. The page of an experiment shows

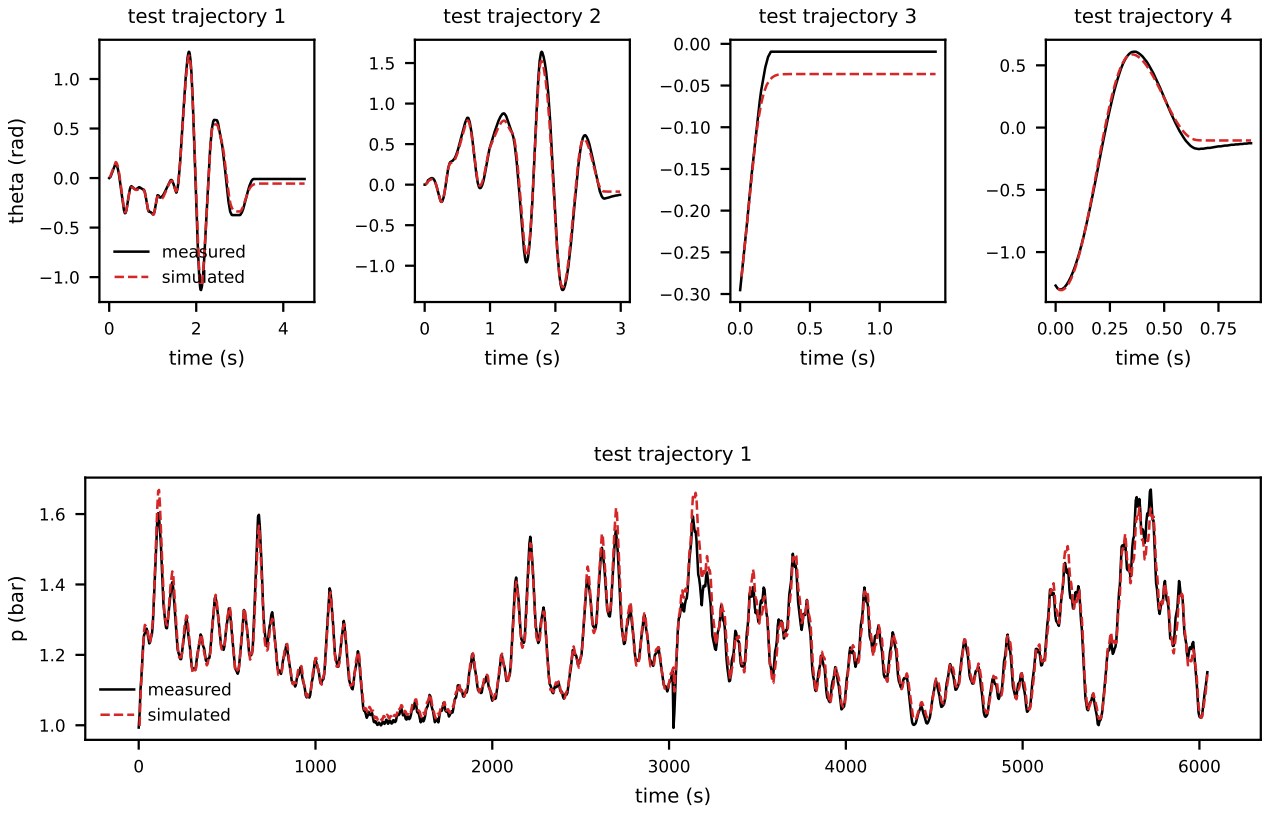
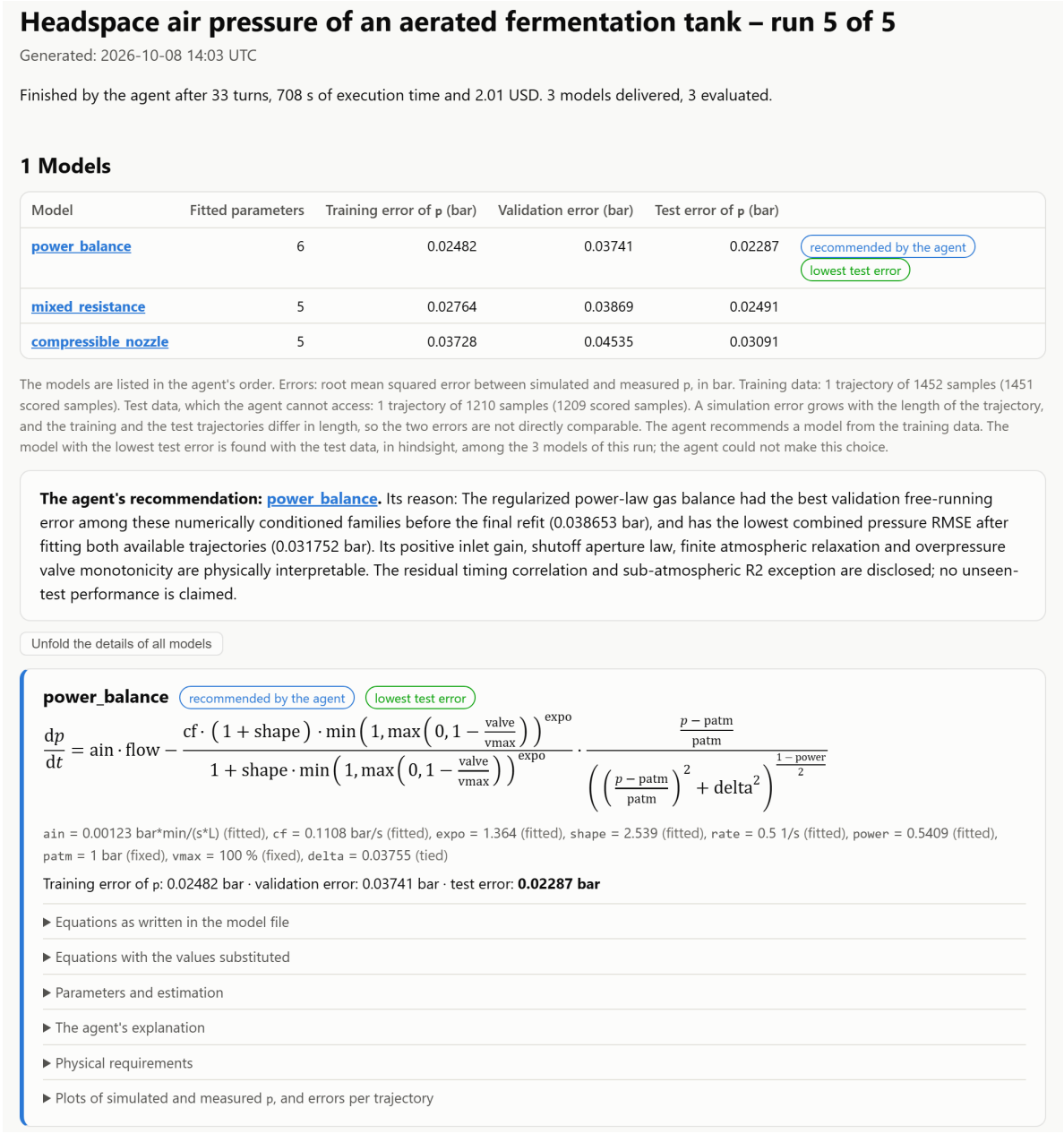


Figure 3: Measured (solid) and simulated (dashed) output on the test trajectories for the recommended model of the run with the median test score, described tasks. Top: friction system, angle θ ($S = 0.0442$ rad); trajectories 1 and 2 are driven, 3 and 4 are free decays. Bottom: fermenter, pressure P ($S = 0.0229$ bar).

the task brief as the agent received it, the settings, and the statistics over the runs with the recommended model of each run. The page of a run (Figure 4) lists the candidates of the pool with their training, validation, and test errors, states the recommendation of the agent with its reason, and shows every candidate as a typeset equation with its parameter values. For each candidate, the reader can unfold the model record, the physical requirements, and the plots of the simulated and the measured output. The page also lists the course of the run turn by turn. Each turn unfolds to the commands of the agent, the files that it wrote, and the outputs that it received, so that a statement of a model record can be compared with what the agent did. The reports of all 30 runs are in the repository and contain the full wording of every model record. We summarize one record here and give the recommended models of the three described tasks in Appendix B.

The example is the record of the recommended candidate of the described task of the fermenter in the run with the median test score ($S = 0.0229$ bar; Figure 4), whose model is equation (B.3) with the parameters of Table B.3 in Appendix B. The model is a balance of the gas in the headspace with six fitted parameters, two fixed parameters, and one tied parameter. The discussion separates the gas balance, which it takes from physics, from the power law of the outflow and the valve characteristic, which it calls modeling choices. It also states that the textbook it names was cited from memory without checking and that no SR package was run. The estimation record reports three stages, each started from the result of the previous one: the error of the supplied derivative on the training data, the simulation error on the training data, and the simulation error on the training and validation data together. It gives the starting values and the bounds as numbers and adds that the final validation error "is not an independent generalization estimate". The limitations address each of the requirements R1 to R5. One parameter shows what the check of the form cannot detect. The relaxation rate ρ is declared as fitted, but its delivered value of $0.5\,\mathrm{s}^{-1}$ equals the upper bound that the agent set so that the numerical integration with the sampling period of 5 s remains stable (Appendix B). The value is therefore determined by a numerical choice of the agent and not by the data alone, which the limitations of the record state.

Table 7 gives the marks of the five items of Section 5.6 for the 30 recommended candidates. Of the 150 marks, 132 are *yes*, 18 *partly* and none *no*; 13 of the 18 concern records of blind tasks.



Headspace air pressure of an aerated fermentation tank – run 5 of 5

Generated: 2026-10-08 14:03 UTC

Finished by the agent after 33 turns, 708 s of execution time and 2.01 USD. 3 models delivered, 3 evaluated.

1 Models

Model	Fitted parameters	Training error of p (bar)	Validation error (bar)	Test error of p (bar)	
power_balance	6	0.02482	0.03741	0.02287	recommended by the agent, lowest test error
mixed_resistance	5	0.02764	0.03869	0.02491	
compressible_nozzle	5	0.03728	0.04535	0.03091	

The models are listed in the agent's order. Errors: root mean squared error between simulated and measured p, in bar. Training data: 1 trajectory of 1452 samples (1451 scored samples). Test data, which the agent cannot access: 1 trajectory of 1210 samples (1209 scored samples). A simulation error grows with the length of the trajectory, and the training and the test trajectories differ in length, so the two errors are not directly comparable. The agent recommends a model from the training data. The model with the lowest test error is found with the test data, in hindsight, among the 3 models of this run; the agent could not make this choice.

The agent's recommendation: power_balance. Its reason: The regularized power-law gas balance had the best validation free-running error among these numerically conditioned families before the final refit (0.038653 bar), and has the lowest combined pressure RMSE after fitting both available trajectories (0.031752 bar). Its positive inlet gain, shutoff aperture law, finite atmospheric relaxation and overpressure valve monotonicity are physically interpretable. The residual timing correlation and sub-atmospheric R2 exception are disclosed; no unseen-test performance is claimed.

Unfold the details of all models

power_balance recommended by the agent · lowest test error

$$\frac{\mathrm{d}p}{\mathrm{d}t} = \mathrm{ain}\cdot\mathrm{flow} - \frac{\mathrm{cf}\cdot\left(1+\mathrm{shape}\right)\cdot\min\left(1,\max\left(0,1-\frac{\mathrm{valve}}{\mathrm{vmax}}\right)\right)^{\mathrm{expo}}}{1+\mathrm{shape}\cdot\min\left(1,\max\left(0,1-\frac{\mathrm{valve}}{\mathrm{vmax}}\right)\right)^{\mathrm{expo}}}\cdot\frac{\frac{p-\mathrm{patm}}{\mathrm{patm}}}{\left(\left(\frac{p-\mathrm{patm}}{\mathrm{patm}}\right)^{2}+\mathrm{delta}^{2}\right)^{\frac{1-\mathrm{power}}{2}}}$$

ain = 0.00123 bar*min/(s*L) (fitted), cf = 0.1108 bar/s (fitted), expo = 1.364 (fitted), shape = 2.539 (fitted), rate = 0.5 1/s (fitted), power = 0.5409 (fitted), patm = 1 bar (fixed), vmax = 100 % (fixed), delta = 0.03755 (tied)

Training error of p: 0.02482 bar · validation error: 0.03741 bar · test error: **0.02287 bar**

▶ Equations as written in the model file

▶ Equations with the values substituted

▶ Parameters and estimation

▶ The agent's explanation

▶ Physical requirements

▶ Plots of simulated and measured p, and errors per trajectory

Figure 4: Report of a run as shown in the browser: the upper part of the page of the run with the median test score of the described task of the fermenter. Top: the candidates of the pool with the number of fitted parameters and the training, validation, and test errors; the labels mark the candidate recommended by the agent and the candidate with the lowest test error. Middle: the recommendation of the agent with its reason. Bottom: the recommended candidate as a typeset equation with its parameter values and their types. The six lines below it unfold to the equations as written by the agent, the parameter table with the estimation record, the discussion, the physical requirements, and the plots. The names of the variables and of the parameters are those of the agent.

Half of the marks *partly* concern item (b): in nine records, the stated origin differs from the record of the run in one claim, for example in the order of two steps, or in a comparison that the run made on a part of the data and the record states without this restriction. Five records give no reason for the starting values of the final estimation stage (item (d)), three do not interpret every term (item (a)), and one declares as fitted a parameter that the script holds at a value chosen from a grid of three values (item (c)). In all 30 records, the delivered parameter values equal the values that the script computes. In the blind tasks, no recommended record states a physical meaning of a term as a fact. The evidence for every mark is in the repository.

In 18 of the 20 runs of the friction system and the fermenter, the agent estimated the recommended model on the training and the validation data together, after the validation data had served to compare structures. The records state

Table 7
Assessment of the model records of the recommended candidates: number of records (of five per experiment) marked yes / partly / no for each item of Section 5.6: (a) interpretation of every term, (b) agreement of the stated origin with the run record, (c) agreement of the declared parameter types with the estimation script, (d) stated reasons for the criterion, the method and the starting values, (e) limitations that follow from the data or the structure.

System	Task	(a)	(b)	(c)	(d)	(e)
Mobile robot	described	5 / 0 / 0	5 / 0 / 0	5 / 0 / 0	5 / 0 / 0	5 / 0 / 0
Mobile robot	blind	5 / 0 / 0	3 / 2 / 0	5 / 0 / 0	4 / 1 / 0	5 / 0 / 0
Friction system	described	4 / 1 / 0	4 / 1 / 0	5 / 0 / 0	5 / 0 / 0	5 / 0 / 0
Friction system	blind	5 / 0 / 0	3 / 2 / 0	4 / 1 / 0	3 / 2 / 0	5 / 0 / 0
Fermenter	described	5 / 0 / 0	2 / 3 / 0	5 / 0 / 0	5 / 0 / 0	5 / 0 / 0
Fermenter	blind	3 / 2 / 0	4 / 1 / 0	5 / 0 / 0	3 / 2 / 0	5 / 0 / 0
All		27 / 3 / 0	21 / 9 / 0	29 / 1 / 0	25 / 5 / 0	30 / 0 / 0

this, and the training errors of Table 5 are computed by the harness on the training data alone.

For the fermenter, all five recommended models of the described task address each of the requirements R1 to R5 in their record and satisfy each of them on the whole grid of Section 5.6. Three records state that R2 does not hold below the equilibrium pressure, where closing the valve reduces the inflow from the atmosphere; the assessment evaluates R2 at pressures of at least 1 bar.

6.4. Cost

A run cost between 0.96 and 3.38 US dollars, the lowest and the highest cost of the 30 runs. Averaged over the five runs of an experiment, a run took 29 to 45 turns and 9 to 21 minutes of execution time, depending on the experiment, and the mean cost of an experiment lies between 1.31 ± 0.25 and 2.70 ± 0.44 US dollars (Table 5). The 30 runs cost 61.10 US dollars, which is 0.48 US dollars per eligible candidate. The recorded cost charges cached input tokens at the full price and is therefore an upper estimate. For comparison, one search of PySR took 4 to 31 minutes on one processor core for one fitting target.

6.5. Interactive case study

This section demonstrates the interaction mode capabilities of ASPIRE on an interactive session. The ASPIRE framework offers a terminal and a browser interface. This session was conducted in the browser interface of the harness on 9 October 2026 and started from the data files of the mobile robot alone. The task interview took 15 turns and 0.46 US dollars; the session, with two rounds and one analysis between them, took 57 turns, 17 minutes of execution time and 5.52 US dollars. The interaction provided the following.

1. *Task specification from data files.* From the training data, the interview agent recognized the trajectories and the probable meaning of the columns, asked six questions about what the data do not determine (Figure 5), and proposed the model form and the evaluation. The resulting specification agrees with the one written by hand (Section 5.1) in the model form, the scored output and the evaluation; the user needs no knowledge of its format.
2. *Questions of the agent.* In round 1, the agent asked about the nature of the measurement error and about the initial states of the intended simulations, each with the modeling decision that it affects. The first answer added that transient effects follow every change of the commands, which the task description did not state; every model record of the round names this among its limitations.
3. *Analysis without a change of the pool.* Asked how much of the error arises after a change of the commands, the agent computed that the first sample after a change, one fifth of the samples, carries 83 % of the squared one-step prediction error of the heading. The user can thus test a conjecture on the data, here the delay stated in Section 5.1, before deciding how to proceed.
4. *Feedback and a further round.* The user asked for expressions without a case distinction and for the evidence for two terms, a cubic term and a dead zone of the angular velocity command. The agent reported that 5 of the 450 training samples lie inside the dead zone, withdrew both models, and delivered a model with two fitted parameters as the recorded successor of the dead-zone model; the pool changed from four models to three. What the user considers readable and defensible thereby shapes the pool.
5. *Changes of the task.* With the feedback, the user added two requirements in natural language, which the two revised model records address; a formal requirement can likewise be relaxed or restored. Requirements that become apparent only when models are seen need no new session.
6. *Labeling of test-informed results.* Round 2 carries the label test-informed. The recommended model and its test error, 0.0437 m, were the same in both rounds: the interaction changed the content of the pool and the understanding of the error, not the accuracy of the recommended model.

Every question, answer, request and change of the task is part of the session record.

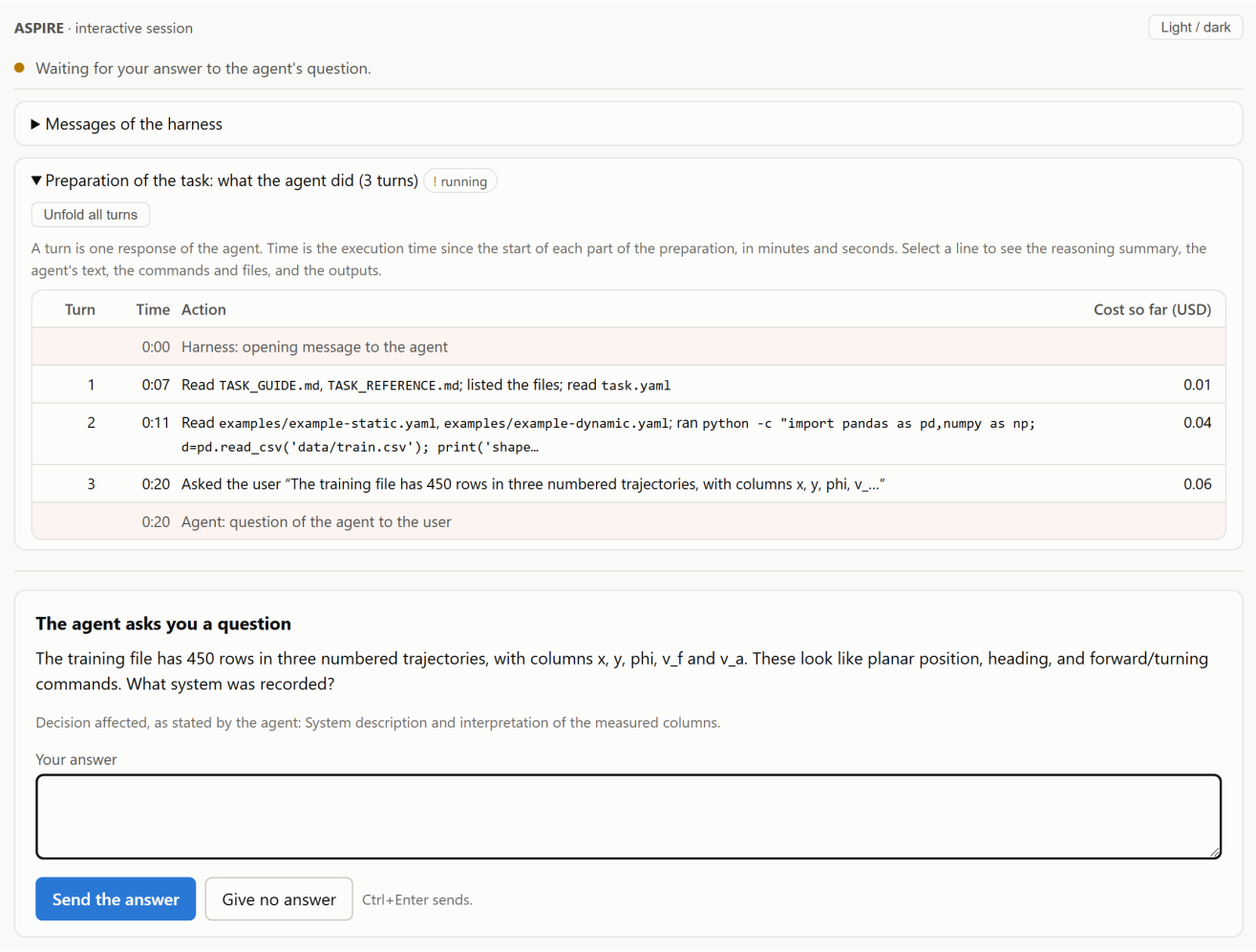


Figure 5: ASPIRE's browser interface during the task interview of the interactive session. Top: the status line, which states what the session waits for. Middle: the course of the interview so far, one line per turn of the interview agent with the execution time and the accumulated cost; in three turns, the agent read the guide to the task specification, two example specifications and the training data, and asked its first question. Bottom: the question, the decision that it affects as stated by the agent, and the form in which the user answers or declines to answer.

7. Discussion

7.1. Effect of the task description

Prior knowledge did not reduce test error. In all three systems, the mean test score of the described task is higher than that of the blind task (Table 5): by 0.0029 m for the mobile robot, by 0.0009 rad for the friction system and by 0.0008 bar for the fermenter. We state these differences as observations on five runs each and on small test sets; for the friction system and the fermenter, the difference is smaller than the sample standard deviation of the blind runs (0.0079 rad and 0.0010 bar).

For the friction system, the five described scores (0.0292 to 0.0459 rad) and the five blind scores (0.0322 to 0.0492 rad) cover nearly the same range. The models differ in form. All five recommended models of the described task contain a mechanism for sticking at rest, which the description and the intended use name, and none of the blind task does; the blind models also violate R2 (Section 6.2). The test score does not show this difference. The test data contain little of what it concerns: the two free decays, in which the system comes to rest, are 230 of the 978 scored samples.

For the mobile robot, the five blind scores (0.0379 to 0.0415 m) all lie below the five described scores (0.0417 to 0.0443 m). A possible explanation is that with the description, the agent recommends the kinematic model with two gains, whereas four of the five blind models contain a dead zone in the response to an input, the term that also lowers the test score in the pools of the described task.

The results of the blind tasks also address the question of recall (Section 5.4). Without any name or description, the agent reached test errors that were not higher, and for two systems, it arrived at the physical structure from the data alone. The low errors, therefore, do not depend on a model

that the language model recalls from a description of the system.

The described models consist of terms with a physical name and a unit, and they have fewer parameters for two of the three systems (2.4 ± 0.9 against 9.6 ± 5.9 for the mobile robot and 5.6 ± 1.1 against 19.0 ± 2.2 for the fermenter). For the fermenter, the described task specifies a physically interpretable model, whereas the blind task produces cubic polynomials with comparable test errors. The described models also satisfy physical properties specified in the task requirements: the direction of the gravitational torque at rest for the friction system (R2), and monotonicity and equilibrium conditions for the fermenter (R1–R5; Section 6.3). These properties are not reflected in the test error and must be assessed separately or specified as formal requirements. The task description also influences the quality of the model records: in the assessment of Section 6.3, 13 of the 18 marks other than *yes* concern blind tasks.

7.2. Reliability of the model records

The harness checks the completeness and consistency of model records but cannot verify their content. We therefore assessed the records manually. None of the 30 records contained an undisclosed discrepancy between the declared parameter types and the estimation scripts. However, nine records contained discrepancies between the reported origin of the model structure and the execution logs. In some cases, the record attributed a modeling choice to an analysis that had actually followed it; in others, it described a comparison more broadly than the executed analysis justified. The records, therefore, documented the methods and results more reliably than the reasoning and sequence of decisions that led to them. The execution logs provide the evidence needed to verify these accounts.

7.3. ASPIRE and a session with a general coding agent

A user can give the same data and the same description to a general coding agent in a conversation, with the same language model and the same tools, and ask for a closed-form model. We have no evidence that the harness changes the error of the identified models. ASPIRE provides a curated interface for agentic system identification; it brings value in aspects such as whether a result can be trusted, compared, and reproduced. The test data are outside the agent's reach, and one implementation computes all errors. The recommendation is fixed before any test result exists. The deliverable has a checked form, and the formal requirements are verified on every model. The complete record of a run is stored. A task is repeated with identical settings, so that the variability of the agent is reported with the result. In a conversation, each of these properties depends on what the user arranges and maintains manually.

7.4. Limitations

The test data are sealed against the agent but not against the authors, who used them during the development of the framework and of the tasks. All runs used one language model, for which the vendor publishes no dated snapshot, so that the results do not establish how ASPIRE performs with other models, and the model behind the identifier may change. While preliminary experiments have shown that ASPIRE also works with different models, it has not been evaluated on a larger scale. Assumptions (A1)–(A3) in Section 3 restrict the framework to a single scored output, to systems whose states are measured, and to a given measurement equation. A score that combines several outputs is not part of the framework.

8. Conclusions

We proposed ASPIRE, a framework for system identification in which an LLM agent selects and executes the identification method and delivers closed-form models from measured data and the prior knowledge that the user provides. Every model found by ASPIRE is accompanied by a record of its parameters, of their estimation, and of its interpretation; the formal physical requirements are verified on it; and its error is computed on test data that the agent cannot access. ASPIRE runs autonomously or in a dialogue with the user.

We evaluated ASPIRE on three measured systems, each with and without a description of the system. All 30 runs delivered valid models, at a cost of 0.96 to 3.38 US dollars per run. For the rotational motion under dry friction, the task stated formal requirements on the direction of the gravitational torque and on the polarity of the motor, and the verification demonstrated no violation in any model found by the agent. Table 6 compares the test errors of the recommended models with those of the models found by PySINDy and PySR. Withholding the description did not increase the test error. It changed the form of the models: with the description, the models consisted of physically named terms and, for two systems, had fewer parameters. In the assessment of the model records of the 30 recommended models, 132 of the 150 marks were *yes*, 18 *partly* and none *no* (Table 7). The records documented the methods and the results more reliably than the sequence of decisions that led to them: nine records differed from the execution logs in one claim on the origin of the model (Section 7.2). In an interactive session on the data of the mobile robot, an analysis requested by the user attributed 83 % of the squared one-step prediction error of the heading to the first sample after a change of the commands, and the user's feedback led to a smaller pool with simpler expressions, while the test error of the recommended model did not change (Section 6.5).

In our future research, we will extend ASPIRE to systems with several outputs and with unmeasured states, compare several language models, and evaluate the interactive operation with users who did not take part in the development of the framework. We will also evaluate how the agent can guide the collection of data in interactive operation: from the training and validation data and the intended use, the agent suggests to the user which data to obtain, so that the operating regions relevant to the problem that the data do not

yet sufficiently cover are covered, and the user adds the new data between the identification rounds. We will also equip the harness with an agent that can assess the compliance of the models with the requirements stated in natural language.

CRediT authorship contribution statement

Erik Derner: Conceptualization, Methodology, Software, Investigation, Writing – original draft. **Jiří Kubalík:** Methodology, Data curation, Writing – review & editing. **Robert Babuška:** Methodology, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) and ChatGPT (OpenAI) to assist with drafting and editing the manuscript text and with implementing the software described herein, under the authors' direction. The inputs of the user in the interactive session of Section 6.5 were drafted with the same assistance from the notes of the author who acted as the user, and were reviewed and adopted by that author. The writing assistance is distinct from the agentic systems that constitute the object of study, whose runs are reported as experimental results. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Acknowledgements

This work was co-funded by the European Union under the project ROBOPROX (reg. no. CZ.02.01.01/00/22_008/0004590). This work has also received funding from the European Union's Horizon Europe research and innovation programme under grant agreement No 101070254 (CoreSense). This work was also supported by the European Union's Horizon Europe research and innovation programme under grant agreement No 101214398 (ELLIOT). Funded by the European Union. Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them.

Data availability

The harness implementation, the task specifications with their training, validation and test data, the experiment configurations, the complete run records of the six experiments and the reports of these experiments as HTML pages are available at `https://github.com/erik-derner/aspire-sysid` under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). The run records contain every request to the language model, every response and tool call, the working directory of the agent with its scripts, the delivered models and their evaluation, together with the software versions and all settings of every run. The repository also contains the scripts, settings and models of the PySINDy and PySR comparisons, the script that computes the reported numbers, tables and figures from the run records, and the assessment of the model records with the evidence for every mark. The test data are sealed against the agent during a run; they are not withheld from readers.

A. System prompt

The harness supplies the following system prompt in autonomous operation. `TASK.md` is the task brief (Section 4.2), and `FORMAT.md` describes the format of the model record and the rules for the agent's texts (Section 4.3). The harness replaces `{packages}` by the names of the SR packages that it finds installed in the agent's environment.

You are an agent for system identification. Your task is in TASK.md in the working directory; read it first. FORMAT.md describes the files you deliver and how to write your texts. A human will assess your models and your explanations.

How you identify the models is your decision. You may read and write files in the working directory, write and run code, fetch web resources, search the web, install Python packages and start helper agents. The installed symbolic-regression packages are {packages}; symbolic-regression search is suited to tasks of this kind, and whether you use it is your decision. Models of any kind, including black-box models, may serve as intermediate tools; the models you deliver have the form that TASK.md requires.

Report what you actually did: the methods, the data and the settings you used. Helper agents cannot start further helpers, and their turns and cost count toward your limits. Everything you do is recorded. The test data and the evaluation results are not available to you; do not try to reach them or any file outside the working directory.

In interactive operation, the following paragraph is appended: *This is an interactive session; TASK.md states how it proceeds. When you ask the user, ask one specific question at a time and say which decision it affects. If the user cannot answer, state the assumption you make instead and its limitation.* The harness adds a description of the environment (the command interpreter and the installed packages with their versions) to the prompt.

B. Recommended models of the described tasks

This appendix gives the recommended candidates of the three described tasks, each from the run with the median

Table B.1
Parameter table of the model (B.1) of the mobile robot as delivered by the agent; values rounded to four significant digits, descriptions quoted and shortened.

Name	Value	Unit	Type	Description
T_s	0.2	s	fixed	"Sampling period `Ts = 0.2` s supplied in TASK.md; used to convert velocity to a one-sample increment."
g_f	0.8508	–	fitted	"Forward command gain; coefficient of `v_f` in effective forward velocity. [...]"
g_a	0.7642	–	fitted	"Angular command gain; coefficient of `v_a` in effective angular velocity. [...]"

test score of the five runs, as the agent delivered them: the expressions and the parameter table, with values rounded to four significant digits and descriptions quoted and shortened. In the quotations, omissions are marked by [...], and names that the agent wrote as code are set in a typewriter font. The complete model records are in the repository.

Mobile robot (test score $S = 0.0428$ m; training error 0.0406 m). The agent names the family "Calibrated unicycle kinematics with forward Euler discretization". The delivered expressions are

$$\begin{aligned} x(k+1) &= x(k) + T_s\, g_f\, v_f(k) \cos\phi(k), \\ y(k+1) &= y(k) + T_s\, g_f\, v_f(k) \sin\phi(k), \\ \phi(k+1) &= \phi(k) + T_s\, g_a\, v_a(k). \end{aligned} \tag{B.1}$$

Table B.1 is its parameter table.

Friction system (test score $S = 0.0442$ rad; training error 0.0616 rad). The agent names the family "Regularized Coulomb-viscous friction with a static dead zone and independent sliding/static speed scales". With the abbreviation $D = au - g\sin\theta$, which the record calls the available acceleration, the delivered expressions are

$$\begin{aligned} \frac{\mathrm{d}\theta}{\mathrm{d}t} &= \omega, \\ \frac{\mathrm{d}\omega}{\mathrm{d}t} &= D - b\,\omega - c\tanh\!\left(\frac{\omega}{v}\right) \\ &\quad - \exp\!\left(-\frac{\omega^2}{v_s^2}\right)\min(\max(D,-c),\,c)\,. \end{aligned} \tag{B.2}$$

At rest, the last term cancels D whenever $|D| \le c$, so that the acceleration is zero over an interval of angles and of voltages. Away from rest, the weight $\exp(-\omega^2/v_s^2)$ of this term vanishes, and the sliding friction $c\tanh(\omega/v)$ remains. The harness proved R1 for this model and found no violation of R2 and R3 at the sampled points. The record derives both requirements from the expression; for R2 it states: "At `omega = 0` and `u = 0`, `D-Min(Max(D,-c),c)` has the same sign as `D`, or is zero; therefore R2 holds." According to the record, the six parameters of Table B.2 were estimated together on the training and validation data by minimizing the squared errors of the simulated angle, from three sets of starting values.

Fermenter (test score $S = 0.0229$ bar; training error 0.0248 bar). The agent names the family "Grey-box nonlinear lumped isothermal gas-inventory model with regularized signed power-law outlet flow and an empirical rational valve characteristic". With the opening of the valve $s = \min(1, \max(0, 1 - u_\mathrm{v}/v_\mathrm{max}))$ and the relative pressure difference $z = (P - p_\mathrm{atm})/p_\mathrm{atm}$, two abbreviations of ours, the delivered expression is

$$\frac{\mathrm{d}P}{\mathrm{d}t} = a_\mathrm{in}\,u_\mathrm{f} - c_f\,\frac{(1+\sigma)\,s^n}{1+\sigma s^n}\,\frac{z}{\left(z^2+\delta^2\right)^{(1-q)/2}}. \tag{B.3}$$

Table B.3 is its parameter table, in which we write the parameters as symbols. The description of ρ in the table is explained as follows; the explanation is ours and is not part of the record. The parameter ρ enters (B.3) only through the tied parameter δ. For a fully open valve ($s = 1$) and a pressure close to the atmospheric pressure ($|z| \ll \delta$), the model reduces to $\mathrm{d}P/\mathrm{d}t \approx a_\mathrm{in}\,u_\mathrm{f} - \rho\,(P - p_\mathrm{atm})$, so that ρ is the rate at which the pressure relaxes to the atmospheric pressure. The harness integrates the model by one step of the fourth-order Runge–Kutta method per sampling period $T_s = 5$ s (Section 4.5). For a linear relaxation with the rate ρ, this integration is stable only for $\rho T_s < 2.785$, that is, for $\rho < 0.557\,\mathrm{s}^{-1}$. The agent bounded ρ by $0.5\,\mathrm{s}^{-1}$, and the delivered value equals this bound.

C. Physical requirements of the fermenter

This appendix gives the physical requirements of the described task of the fermenter. The statements R1 to R5 are quoted in full, as the agent received them.

R1 "Monotonicity with respect to inlet flow. At a fixed pressure and valve position, increasing the inlet air flow rate cannot decrease the pressure derivative."

R2 "Monotonicity with respect to valve closure. At a fixed pressure and inlet flow rate, closing the outlet valve further cannot decrease the pressure derivative."

R3 "Zero-flow equilibrium. With zero inlet flow and an open outlet valve, atmospheric pressure (approximately 1 bar) is the pressure equilibrium."

R4 "Pressure relaxation. With zero inlet flow and an open outlet valve, pressure tends toward atmospheric pressure."

R5 "Fully closed valve. For a perfectly sealed tank with a fully closed outlet valve, the pressure cannot decrease when air is supplied."

Table B.2
Parameter table of the model (B.2) of the friction system as delivered by the agent; values rounded to four significant digits, descriptions quoted.

Name	Value	Unit	Type	Description
a	28.52	rad/(s^2 V)	fitted	"Motor-voltage to acceleration gain `a`; only gain divided by inertia is identified."
g	112.1	rad/s^2	fitted	"Positive restoring-acceleration scale `g`."
b	1.591	1/s	fitted	"Positive viscous acceleration coefficient `b`; includes motor back-emf damping and bearing friction without separating them."
c	22.22	rad/s^2	fitted	"Dry-friction acceleration scale `c`."
v	0.5181	rad/s	fitted	"Sliding-friction regularization speed `v` in `tanh(omega/v)`."
v_s (`vs`)	0.1645	rad/s	fitted	"Independent static-friction localization speed `vs` in `exp(-(omega/vs)**2)`."

Table B.3
Parameter table of the model (B.3) of the fermenter as delivered by the agent; values rounded to four significant digits, descriptions quoted and shortened. The names of the agent are given in parentheses. The tied parameter δ is computed by the equation $\delta = (c_f/(p_{\mathrm{atm}}\,\rho))^{1/(1-q)}$, which the record states. The description of ρ is explained in the text.

Name	Value	Unit	Type	Description
a_{in} (`ain`)	1.230×10^{-3}	bar min/(s L)	fitted	"Inlet conversion gain `ain`: pressure rate per measured `flow` [...]"
c_f (`cf`)	0.1108	bar/s	fitted	"Full-open outlet scale `cf` in bar/s for the dimensionless pressure-driving law. [...]"
n (`expo`)	1.364	–	fitted	"Positive aperture exponent `expo` [...]. It describes the unknown actuator-to-effective-opening characteristic; it is not a measured valve geometry."
σ (`shape`)	2.539	–	fitted	"Nonnegative series-loss shape `shape` [...]. The rational aperture law saturates as the opening grows [...]"
ρ (`rate`)	0.5000	1/s	fitted	"Full-open atmospheric small-signal relaxation rate `rate` [...]. The upper bound of `rate = 0.5` 1/s was chosen for stable five-second RK4 relaxation [...]; the final estimate reaches that bound."
q (`power`)	0.5409	–	fitted	"Far-from-atmosphere pressure exponent `power` [...]. It interpolates empirical fractional pressure-loss behavior and is not assumed to equal an isentropic exponent."
p_{atm} (`patm`)	1	bar	fixed	"Atmospheric reference `patm = 1` bar, taken from the approximately one-bar equilibrium in TASK.md and deliberately not estimated; it fixes the equilibrium rather than absorbing pressure-sensor bias."
$v_{\max}$ (`vmax`)	100	%	fixed	"Fully closed calibration `vmax = 100` percent, given in TASK.md [...]"
δ (`delta`)	0.03755	–	tied	"Dimensionless near-atmospheric pressure scale `delta`; computed from the outlet scale and the fitted atmospheric relaxation rate. This gives a finite near-equilibrium slope rather than a singular fractional-power slope."

References

[1] Abhyankar, N., Li, S., Kabra, S., Ramakrishnan, N., Gel, Y., Reddy, C.K., 2026. LLM-ACES: Closed-loop discovery of dynamical systems with LLM-guided adaptive search. arXiv preprint. doi:10.48550/arXiv.2606.25039, arXiv:2606.25039.

[2] Babuška, R., 2012. Fuzzy modeling for control. Springer Science & Business Media.

[3] Brunton, S.L., Proctor, J.L., Kutz, J.N., 2016. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. Proceedings of the National Academy of Sciences 113, 3932–3937. doi:10.1073/pnas.1517384113.

[4] Chan, J.S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., Weng, L., Mądry, A., 2025. MLE-bench: Evaluating machine learning agents on machine learning engineering, in: International Conference on Learning Representations (ICLR), pp. 50466–50494. arXiv:2410.07095.

[5] Cornelio, C., Dash, S., Austel, V., Josephson, T.R., Goncalves, J., Clarkson, K.L., Megiddo, N., El Khadir, B., Horesh, L., 2023. Combining data and theory for derivable scientific discovery with AI-Descartes. Nature Communications 14, 1777. doi:10.1038/s41467-023-37236-y.

[6] Cory-Wright, R., Cornelio, C., Dash, S., El Khadir, B., Horesh, L., 2024. Evolving scientific discovery by unifying data and background knowledge with AI Hilbert. Nature Communications 15, 5922. doi:10.1038/s41467-024-50074-w, arXiv:2308.09474.

[7] Cranmer, M., 2023. Interpretable machine learning for science with PySR and SymbolicRegression.jl. arXiv preprint. doi:10.48550/arXiv.2305.01582, arXiv:2305.01582.

[8] d'Ascoli, S., Becker, S., Mathis, A., Schwaller, P., Kilbertus, N., 2024. ODEFormer: Symbolic regression of dynamical systems with transformers, in: International Conference on Learning Representations (ICLR), pp. 21943–21976. arXiv:2310.05573.

[9] Derner, E., Kubalík, J., Babuška, R., 2021. Guiding robot model construction with prior features, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 7112–7118. doi:10.1109/IROS51168.2021.9635831.

[10] Du, M., Chen, Y., Wang, Z., Nie, L., Zhang, D., 2024. Large language models for automatic equation discovery of nonlinear dynamics. Physics of Fluids 36, 097121. doi:10.1063/5.0224297, arXiv:2405.07761.

[11] Grayeli, A., Sehgal, A., Costilla-Reyes, O., Cranmer, M., Chaudhuri, S., 2024. Symbolic regression with a learned concept library, in: Advances in Neural Information Processing Systems (NeurIPS), Curran Associates, Inc.. pp. 44678–44709. doi:10.52202/079017-1419, arXiv:2409.09359.

[12] Grosnit, A., Maraval, A., Refinath, S.N., Zhao, Z., Doran, J., Paolo, G., Thomas, A., Gonzalez, J., Kumar, A., Khandelwal, K., Benechehab, A., Cherkaoui, H., Attia El-Hili, Y., Shao, K., Hao, J., Yao, J., Kégl, B., Bou-Ammar, H., Wang, J., 2024. Kolb-based experiential learning for generalist agents with human-level kaggle data science performance. arXiv preprint. doi:10.48550/arXiv.2411.03562, arXiv:2411.03562.

[13] Guo, S., Deng, C., Wen, Y., Chen, H., Chang, Y., Wang, J., 2024. DS-Agent: Automated data science by empowering large language models with case-based reasoning, in: Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR. pp. 16813–16848. arXiv:2402.17453.

[14] He, H., Liu, X., Kuhlman, C.J., Deng, X., 2026. An experimental design approach to evaluating agentic AI's autonomous model discovery. arXiv preprint. doi:10.48550/arXiv.2607.06413, arXiv:2607.06413.

[15] Ifargan, T., Hafner, L., Kern, M., Alcalay, O., Kishony, R., 2025. Autonomous LLM-driven research – from data to human-verifiable research papers. NEJM AI 2, AIoa2400555. doi:10.1056/AIoa2400555, arXiv:2404.17605.

[16] Jansen, P., Tafjord, O., Radensky, M., Siangliulue, P., Hope, T., Dalvi Mishra, B., Majumder, B.P., Weld, D.S., Clark, P., 2025. CodeScientist: End-to-end semi-automated scientific discovery with code-based experimentation, in: Findings of the Association for Computational Linguistics (ACL), Association for Computational Linguistics. pp. 13370–13467. doi:10.18653/v1/2025.findings-acl.692, arXiv:2503.22708.

[17] Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., Wu, Y., 2025. AIDE: AI-driven exploration in the space of code. arXiv preprint. doi:10.48550/arXiv.2502.13138, arXiv:2502.13138.

[18] Kon, P.T.J., Liu, J., Ding, Q., Qiu, Y., Yang, Z., Huang, Y., Srinivasa, J., Lee, M., Chowdhury, M., Chen, A., 2025. Curie: Toward rigorous and automated scientific experimentation with AI agents. arXiv preprint. doi:10.48550/arXiv.2502.16069, arXiv:2502.16069.

[19] Kubalík, J., Babuška, R., 2025. Neuro-evolutionary approach to physics-aware symbolic regression, in: Genetic and Evolutionary Computation Conference (GECCO), pp. 1264–1272. doi:10.1145/3712256.3726434.

[20] Kubalík, J., Derner, E., Babuška, R., 2020. Symbolic regression driven by training data and prior knowledge, in: Genetic and Evolutionary Computation Conference (GECCO), pp. 958–966. doi:10.1145/3377930.3390152.

[21] Kubalík, J., Derner, E., Babuška, R., 2021. Multi-objective symbolic regression for physics-aware dynamic modeling. Expert Systems with Applications 182, 115210. doi:10.1016/j.eswa.2021.115210.

[22] Kubalík, J., Derner, E., Babuška, R., 2023. Toward physically plausible data-driven models: A novel neural network approach to symbolic regression. IEEE Access 11, 61481–61501. doi:10.1109/ACCESS.2023.3287397.

[23] Kubalík, J., Fučelová, N., Babuška, R., 2026. Asynchronous neuro-evolutionary symbolic regression with maturity-based replacement, in: Proceedings of the Genetic and Evolutionary Computation Conference (GECCO), pp. 984–992. doi:10.1145/3795095.3805164.

[24] La Cava, W., Orzechowski, P., Burlacu, B., Olivetti de França, F., Virgolin, M., Jin, Y., Kommenda, M., Moore, J.H., 2021. Contemporary symbolic regression methods and their relative performance, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, Curran Associates, Inc.. pp. 695–710. arXiv:2107.14351.

[25] Leontaritis, I.J., Billings, S.A., 1985. Input-output parametric models for non-linear systems Part I: Deterministic non-linear systems. International Journal of Control 41, 303–328. doi:10.1080/0020718508961129.

[26] Lu, C., Lu, C., Lange, R.T., Yamada, Y., Hu, S., Foerster, J., Ha, D., Clune, J., 2026a. Towards end-to-end automation of AI research. Nature 651, 914–919. doi:10.1038/s41586-026-10265-5.

[27] Lu, R., Wu, Y., Kou, E., Fu, L., Xiao, W., Mandlekar, A., Xu, Y., Shi, G., Goldberg, K., Chen, A., Chowdhury, M., Zhu, Y., Fan, L., Wang, G., 2026b. ASPIRE: Agentic /skills discovery for robotics. arXiv preprint. doi:10.48550/arXiv.2607.00272, arXiv:2607.00272.

[28] Merler, M., Haitsiukevich, K., Dainese, N., Marttinen, P., 2024. In-context symbolic regression: Leveraging large language models for function discovery, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop), Association for Computational Linguistics. pp. 427–444. doi:10.18653/v1/2024.acl-srw.49, arXiv:2404.19094.

[29] Mežnar, S., Todorovski, L., Džeroski, S., 2026. SRToolkit: Shared infrastructure for symbolic regression research. Workshop on Symbolic Regression and Equation Discovery, IEEE World Congress on Computational Intelligence (WCCI). Software: https://github.com/smeznar/SymbolicRegressionToolkit.

[30] Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.S., Wagner, A.Z., Shirobokov, S., Kozlovskii, B., Ruiz, F.J.R., Mehrabian, A., Kumar, M.P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., Balog, M., 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint. doi:10.48550/arXiv.2506.13131, arXiv:2506.13131.

[31] OpenAI, 2026. GPT-6.1 Sol: Model documentation. https://developers.openai.com/api/docs/models/gpt-6.1-sol. Accessed 9 October 2026.

[32] Piga, D., Forgione, M., 2026. ASIA: An autonomous system identification agent. IEEE Control Systems Letters 10, 2251–2256. doi:10.1109/LCSYS.2026.3717047, arXiv:2605.10480.

[33] Raptis, E.K., Kapoutsis, A.C., Kosmatopoulos, E.B., 2025. Agentic LLM-based robotic systems for real-world applications: A review on their agenticness and ethics. Frontiers in Robotics and AI 12, 1605405. doi:10.3389/frobt.2025.1605405.

[34] Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., Kohli, P., Fawzi, A., 2024. Mathematical discoveries from program search with large language models. Nature 625, 468–475. doi:10.1038/s41586-023-06924-6.

[35] Schoukens, M., Champneys, M., Beintema, G.I., Rogers, T.J., 2026. Benchmarking for nonlinear system identification: Submission platform and baseline results. Data-Centric Engineering 7, e40. doi:10.1017/dce.2026.10067.

[36] Shojaee, P., Meidani, K., Gupta, S., Barati Farimani, A., Reddy, C.K., 2025. LLM-SR: Scientific equation discovery via programming with large language models, in: International Conference on Learning Representations (ICLR), pp. 16054–16085. arXiv:2404.18400.

[37] de Silva, B.M., Champion, K., Quade, M., Loiseau, J.C., Kutz, J.N., Brunton, S.L., 2020. PySINDy: A python package for the sparse identification of nonlinear dynamical systems from data. Journal of Open Source Software 5, 2104. doi:10.21105/joss.02104.

[38] Stephens, T., 2026. gplearn: Genetic programming in Python, with a scikit-learn inspired API. Version 0.4.3 [software]. https://github.com/trevorstephens/gplearn.

[39] Su, J., Tu, S., Sun, B., Liang, X., 2026. STRIDE: A self-reflective agent framework for reliable automatic equation discovery. arXiv preprint. doi:10.48550/arXiv.2605.17790, arXiv:2605.17790.

[40] Van Can, H.J.L., Te Braake, H.A.B., Hellinga, C., Krijgsman, A.J., Verbruggen, H.B., Luyben, K.C.A.M., Heijnen, J.J., 1995. Design and real time testing of a neural model predictive controller for a nonlinear system. Chemical Engineering Science 50, 2419–2430. doi:10.1016/0009-2509(95)00083-H.

[41] Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright,

J., van der Walt, S.J., Brett, M., Wilson, J., Millman, K.J., Mayorov, N., Nelson, A.R.J., Jones, E., Kern, R., Larson, E., Carey, C.J., Polat, İ., Feng, Y., Moore, E.W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E.A., Harris, C.R., Archibald, A.M., Ribeiro, A.H., Pedregosa, F., van Mulbregt, P., SciPy 1.0 Contributors, 2020. SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods 17, 261–272. doi:10.1038/s41592-019-0686-2.

[42] Wang, R., Wang, B., Li, K., Zhang, Y., Cheng, J., 2025. DrSR: LLM based scientific equation discovery with dual reasoning from data and experience. arXiv preprint. doi:10.48550/arXiv.2506.04282, arXiv:2506.04282.

[43] Xia, S., Sun, Y., Liu, P., 2026. SR-Scientist: Scientific equation discovery with agentic AI, in: International Conference on Learning Representations (ICLR), pp. 75787–75811. arXiv:2510.11661.

[44] Xiao, J., Chen, X., Peng, J., Wang, Q., Jia, M., Lai, Z., Yu, G., Li, D., Li, T., Liu, J., 2026. Prior-guided symbolic regression: Towards scientific consistency in equation discovery. arXiv preprint. doi:10.48550/arXiv.2602.13021, arXiv:2602.13021.

[45] Xin, A., Siow, J., Wang, J., Yao, Z., Zhang, F., Song, J., Hou, L., Li, J., 2026. EurekAgent: Agent environment engineering is all you need for autonomous scientific discovery. arXiv preprint. doi:10.48550/arXiv.2606.13662, arXiv:2606.13662.

[46] Yamada, Y., Lange, R.T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., Ha, D., 2025. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. arXiv preprint. doi:10.48550/arXiv.2504.08066, arXiv:2504.08066.

[47] Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O., 2024. SWE-agent: Agent-computer interfaces enable automated software engineering, in: Advances in Neural Information Processing Systems (NeurIPS), Curran Associates, Inc.. pp. 50528–50652. doi:10.52202/079017-1601, arXiv:2405.15793.

[48] Yang, J., Venkatachalam, O., Kianezhad, M., Vadgama, S., Yu, R., 2026. Think like a scientist: Physics-guided LLM agent for equation discovery. arXiv preprint. doi:10.48550/arXiv.2602.12259, arXiv:2602.12259.

[49] Yang, X., Yang, X., Fang, S., Zhang, Y., Wang, J., Xian, B., Li, Q., Li, J., Xu, M., Li, Y., Pan, H., Zhang, Y., Liu, W., Shen, Y., Chen, W., Bian, J., 2025. R&D-Agent: An LLM-agent framework towards autonomous data science. arXiv preprint. doi:10.48550/arXiv.2505.14738, arXiv:2505.14738.

[50] Zhang, H., Chen, Q., Xue, B., Banzhaf, W., Zhang, M., 2027. Benchmarking zero-shot LLM-generated parent selection in genetic programming for symbolic regression, in: Parallel Problem Solving from Nature – PPSN XIX, Lecture Notes in Computer Science, vol. 16989, Springer, Cham. pp. 233–248. doi:10.1007/978-3-032-36217-9_15, arXiv:2607.23505.

[51] Ziaei Bideh, A., Gryak, J., 2026. LLM-ODE: Data-driven discovery of dynamical systems with large language models, in: Genetic and Evolutionary Computation Conference (GECCO), pp. 853–860. doi:10.1145/3795095.3805067.